

**VISION-DRIVEN AUTONOMOUS PCB ASSEMBLY
PROTOTYPE
A PROJECT REPORT**

Submitted by

AKASH M	(711622106002)
ARJUN V	(711622106005)
GOWTHAM B	(711622106018)
NIVESH S	(711622106034)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING (B.E)

in

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**



KATHIR COLLEGE OF ENGINEERING

(Autonomous)

ANNA UNIVERSITY: CHENNAI 600 025

APRIL 2026

KATHIR COLLEGE OF ENGINEERING, COIMBATORE
(Autonomous)

BONAFIDE CERTIFICATE

Certified that this project report " **VISION-DRIVEN AUTONOMOUS PCB ASSEMBLY PROTOTYPE**" is the Bonafide work of

AKASH M	(711622106002)
ARJUN V	(711622106005)
GOWTHAM B	(711622106018)
NIVESH S	(7116221060034)

who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of other dissertation on the basics of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

.....
Dr. N.K. Anushkannan, M.E, Ph.D.,
HEAD OF THE DEPARTMENT
Professor & Head,
Department of ECE,
Kathir College of Engineering,
Neelambur,
Coimbatore-641062.

SIGNATURE

.....
Mrs. S. SANJULA.
SUPERVISOR
Assistant Professor,
Department of ECE
Kathir College of Engineering,
Neelambur,
Coimbatore-641062.

The candidates with university register number was
examined in the Project viva-voce examination held on.....

.....
INTERNAL EXAMINER

.....
EXTERNAL EXAMINER

ACKNOWLEDGEMENT

The success of a work depends on a team and cooperation. We take this opportunity to express our gratitude and thanks to everyone who helped us in our project. We would like to thank the management for excellent facilities and the constant support provided by them to complete this project.

It is indeed our great honor and bounded duty to thank our Respected Chairman **Thiru. E. S. KATHIR**, for his academic interest shown towards the students. We are indebted to our beloved Secretary **Tmt. LAVANYA KATHIR**, for motivating and providing us with all facilities. We wish to express our sincere regards and deep sense of gratitude to **Dr. R. UDAIYAKUMAR, M.E, Ph.D.**, Principal, for the excellent facilities and encouragement provided during the course of the study and project.

We are indebted to **Dr. N. K. ANUSHKANNAN, M.E, Ph.D.**, Professor & Head/ECE, for having permitted us to carry out this project and giving the complete freedom to utilize the resources of the department.

We express our sincere thanks to our project guide **Mrs. S. SANJULA**, Assistant Professor/ECE, for her valuable guidance and encouragement given to us for this project. Her expertise in embedded systems and computer vision proved invaluable in shaping the direction of our research.

We solemnly express our thanks to all the teaching and non-teaching staff of the Electronics and Communications Engineering Department, family and friends for their valuable support which inspired us to work on this project.

ABSTRACT

The growing use of automated systems in printed circuit board (PCB) assembly enables manufacturers to achieve better placement accuracy and reduced reliance on manual labour in production activities. However, industrial pick-and-place machines face major challenges due to their high setup costs and the need for complex control systems, which limit their accessibility for research applications and early-stage product development. To address this, this research introduces a budget-friendly vision-based system for PCB component assembly using basic computer vision techniques and wireless motion control. The system employs a fixed camera setup to capture PCB images and identify placement markings through grayscale conversion, noise removal, adaptive thresholding, and contour-based feature extraction.

The identified placement coordinates are transmitted via serial communication to an embedded controller, which drives a stepper motor-based XY gantry positioning system. A standard mechanical gripper is used to perform pick-and-place operations. During prototype testing, challenges such as lighting variations and mechanical vibrations were encountered and mitigated through calibration techniques. Experimental results demonstrate that the system achieves precise placement for through-hole components with an accuracy of ± 2 mm in controlled conditions. The proposed system is modular, cost-effective (under Rs. 20,000), and suitable for educational purposes, offering a practical demonstration of automated PCB assembly. It achieves a component detection rate of 92% and repeatability of 90% across multiple PCB samples.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	iii
	LIST OF FIGURES	ix
	LIST OF TABLES	xi
	LIST OF SYMBOLS	xii
1	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 EMBEDDED SYSTEMS	2
	1.2.1 Definition	2
	1.2.2 Key Characteristics	2
	1.2.3 Applications	3
	1.2.4 Microcontroller	4
	1.3 COMPUTER VISION	5
	1.3.1 Definition	5
	1.3.2 Key Characteristics	5
	1.3.3 Applications	6
	1.4 EMBEDDED SYSTEM IN COMPUTER VISION	7
	1.4.1 Integration	7
	1.4.2 Advantages	8
	1.5 PRINTED CIRCUIT BOARD (PCB)	9
	1.5.1 Definition	9
	1.5.2 Structure Of Pcb	9

	1.5.3 Types Of Pcb Components	9
	1.5.4 Pcb Assembly Process	10
	1.6 ANALYSIS ON PCB MANUFACTURING INDUSTRY	11
	1.7 SUMMARY	11
2	LITERATURE SURVEY	13
	2.1 INTRODUCTION	13
	2.2 EXISTING APPROACHES	13
	2.3 PROBLEM STATEMENT	18
	2.4 RESEARCH GAP	19
	2.5 OBJECTIVES	20
	2.6 SUMMARY	20
3	METHODOLOGY	21
	3.1 SYSTEM ARCHITECTURE	21
	3.2 PROPOSED BLOCK DIAGRAM	22
	3.3 HARDWARE DESIGN	23
	3.3.1 Esp8266 Node (Microcontroller Selection)	23
	3.3.2 Nema 17 Stepper Motor (Motor Control System)	24
	3.3.3 Usb 8Mp Camera Module (Vision System)	24
	3.3.4 TMC2209 Motor Drivers	26
	3.3.5 T8 Lead Screw Assembly	27
	3.3.6 Mechanical Gripper	28
	3.4 MECHANICAL ASSEMBLY	28
	3.4.1 XY Gantry Structure	28
	3.4.2 Assembly Process	29
	3.5 ELECTRICAL CONNECTIONS	30

3.5.1 Power Supply	30
3.5.2 Motor Driver Wiring	30
3.5.3 Bill of Materials	31
3.6 SUMMARY	32
3.7 SOFTWARE DESIGN	33
3.7.1 Vision Processing Pipeline	33
3.7.2 Coordinate Transformation	34
3.7.3 Communication Protocol	35
3.8 SYSTEM WORKFLOW	35
3.9 SUMMARY	37
3.10 SYSTEM SOFTWARE ARCHITECTURE AND DEVELOPMENT SETUP	37
3.10.1 Python And Opencv	37
3.10.2 Adruino IDE	38
3.11 VISION PROCESSING IMPLEMENTATION	39
3.11.1 Image Acquisition	39
3.11.2 Image Preprocessing	40
3.11.3 Contour Detection And Filtering	41
3.11.4 Centroid Calculation	41
3.12 EMBEDDED FIRMWARE	42
3.12.1 System Initialization	42
3.12.2 Coordinate Reception	42
3.12.3 Motion Control	43
3.13 COMMUNICATION PROTOCOL	43

	3.13.1 Message Format	43
	3.13.2 Handshaking	44
	3.14 SUMMARY	44
4	SOFTWARE IMPLEMENTATION	45
	4.1 DEVELOPMENT ENVIRONMENT	45
	4.1.1 Software Framework and Implementation Environment	45
	4.2 SOFTWARE TOOLS AND LIBRARIES	45
	4.2.1 Python 3.x	45
	4.2.2 OpenCV (Open Source Computer Vision Library)	46
	4.2.3 NumPy	48
	4.2.4 Pyserial	48
	4.2.5 Arduino IDE	49
	4.2.6 Accelstepper Library	50
	4.2.7 Esp8266 Board Package	50
	4.2.8 OpenCV's Calibrate Camera Function	51
5	RESULTS AND DISCUSSION	52
	5.1 TEST SETUP	52
	5.2 PERFORMANCE METRICS	52
	5.2.1 Placement Accuracy	52
	5.2.2 Detection Rate	52
	5.2.3 Cycle Time	53
	5.3 TEST RESULTS	53
	5.4 COMPARATIVE ANALYSIS	54
	5.5 ERROR ANALYSIS	55

	5.5.1 Sources of Error	55
	5.5.2 Error Mitigation	55
	5.6 IMPACT OF PCB ASSEMBLY AUTOMATION	56
	5.7 DISCUSSION	56
	5.8 SUMMARY	56
6	CONCLUSION	57
	6.1 CONCLUSION	57
	6.2 FUTURE SCOPE	57
	6.3 SUMMARY	57
	REFERENCES	59
	APPENDIX	62

LIST OF FIGURES

FIGURE	NAME	PAGE NO
1.1	Model Of Vision-Driven Autonomous Pcb Assembly	1
1.2	Embedded System	2
1.3	Pick And Place Robotic Arm	3
1.4	Application Of Embedded System	6
1.5	Microcontroller With Camara Module	7
1.6	Printed Circuit Board	9
1.7	Surface Mount Technology	10
3.1	System Architecture	21
3.2	Block Diagram Of Working Process	22
3.3	Esp8266 Node Mcu Microcontroller	23
3.4	Nema 17 Stepper Motor	24
3.5	Usb 8mp Camara Module	25
3.6	Pcb Image Processing	26
3.7	Tmc2209 Motor Driver	26
3.8	T8 Lead Screw	27
3.9	Structure Of Xy Gantry	28
3.10	Software Design Flow Chart	34
3.11	Automated Pcb Assembly Flow Chart	36
3.12	Python And Open Cv	37
3.13	Arduino Ide	38
3.14	Original Image Of Pcb Silk Screen Symbols	39
3.15	Image Acquisition	39
3.16	Image Processing	40

3.17	Contour Detection And Filtering Image	41
4.1	Python 3.X	45
4.2	Open Cv	46
4.3	Numpy	48
4.4	Pvserial	48
4.6	Arduino Ide	49
4.7	Esp8266 Board Package	50

LIST OF TABLES

TABLE	NAME	PAGE NO
1.1	Features of Microcontroller	4
3.1	Cost Of Components	31
5.1	Performance Test Results	53
5.2	Comparative Analysis	54
6.1	System Specification	63

LIST OF SYMBOLS AND ABBREVIATIONS

IoT	Internet of Things
PCB	Printed Circuit Board
SMT	Surface Mount Technology
GPS	Global Positioning System
GSM	Global System for Mobile Communications
LCD	Liquid Crystal Display
LED	Light Emitting Diode
ADC	Analog to Digital Converter
RAM	Random Access Memory
ROM	Read Only Memory
CPU	Central Processing Unit
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
TX	Transmit
RX	Receive
I2C	Inter-Integrated Circuit
SPI	Serial Peripheral Interface
Wi-Fi	Wireless Fidelity
OpenCV	Open Source Computer Vision Library
XY	Two-Dimensional Coordinate Axes
NEMA	National Electrical Manufacturers Association
IDE	Integrated Development Environment
API	Application Programming Interface
AOI	Automated Optical Inspection

YOLO	You Only Look Once
CAD	Computer-Aided Design
CAM	Computer-Aided Manufacturing
EDM	Electrical Discharge Machining
WEDM	Wire Electrical Discharge Machining
SMPS	Switched-Mode Power Supply
PID	Proportional-Integral-Derivative
PWM	Pulse Width Modulation
GPIO	General Purpose Input/Output
ACK	Acknowledgment
UDP	User Datagram Protocol
TCP/IP	Transmission Control Protocol/Internet Protocol
ROI	Region of Interest
FPS	Frames Per Second
BOM	Bill of Materials
QC	Quality Control

INTRODUCTION

1.1 BACKGROUND

Electronics technology is growing very fast and has improved modern manufacturing. PCB assembly is an important process that needs high accuracy and speed. As devices become smaller and more complex, automation is becoming necessary. Embedded systems and computer vision help in this process. Embedded systems control the hardware, while computer vision helps to detect components and check their position. Together, they make the system faster, smarter, and more accurate.

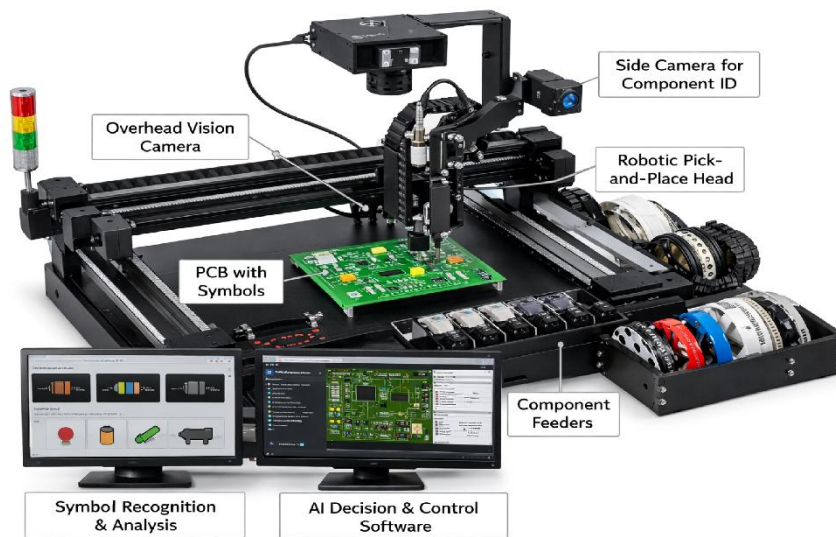


Figure 1.1: Model Of Vision-Driven Autonomous Pcb Assembly

Model Of Vision-Driven Autonomous Pcb Assembly as shown in Fig 1.1. Manual PCB assembly is slow and can lead to errors. It also needs skilled workers, and mistakes can affect the final product. Industrial SMT machines are very accurate and fast, but they are expensive and hard to set up. So, this project proposes a low-cost system using a camera, OpenCV, and an ESP8266 with an XY gantry. The system detects PCB points, converts them into coordinates, and moves motors to place components. The total cost is under Rs. 20,000, making it useful for learning and small projects.

1.2 EMBEDDED SYSTEMS

1.2.1 Definition

Embedded systems are specialized computing units designed to perform dedicated tasks within larger frameworks. Unlike general-purpose computers, they are optimized for specific operations using a combination of microcontrollers, sensors, and real-time software. This compact architecture ensures efficient resource management, making them ideal for environments where power, space, and cost are limited.

The primary distinction lies in design philosophy: general computers prioritize flexibility and multitasking, while embedded systems focus on deterministic, predictable performance. By stripping away unnecessary functions, these systems achieve high reliability and superior efficiency for their intended use. This specialization allows them to deliver consistent results while maintaining a minimal hardware footprint. Embedded System as shown in Fig 1.2.

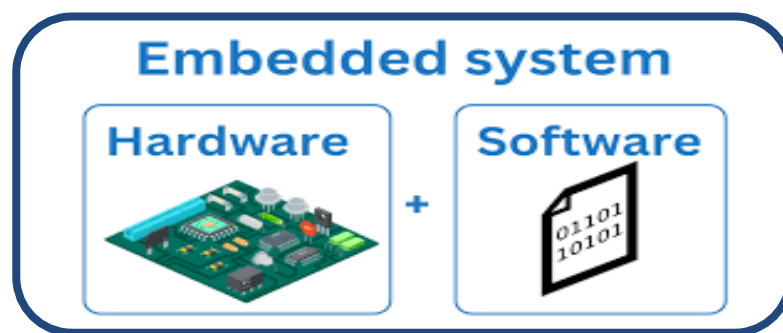


Figure 1.2: Embedded System

1.2.2 Key Characteristics

Embedded systems exhibit several distinctive characteristics that make them suitable for dedicated applications. Real-time operation is perhaps the most critical feature, ensuring immediate response to inputs which is essential in applications like medical devices, automotive control systems, and industrial automation. The system must process data and generate outputs within strict timing constraints to maintain system stability and safety.

Resource constraints are another defining characteristic. Embedded systems typically operate with limited processing power, memory, and storage compared to general-purpose computers. These limitations drive efficient software design and careful hardware selection.

Developers must optimize code for size and speed while ensuring reliable operation under resource constraints.

Power efficiency is crucial for battery-powered and portable embedded devices. Low power consumption extends battery life and reduces heat generation, enabling compact designs without elaborate cooling systems. Modern microcontrollers incorporate various power-saving modes and efficient clock management to minimize energy consumption.

Reliability and stability are paramount in embedded systems, as many applications require continuous operation without failure. Medical devices, automotive systems, and industrial controllers must function reliably over extended periods, often in harsh environmental conditions. Robust hardware design, careful software development, and thorough testing ensure system reliability.

1.2.3 Applications

Embedded systems have become essential across nearly every industry, enabling automation, real-time processing, and intelligent control in a wide range of applications. In the automotive sector, they play a critical role in engine management, anti-lock braking systems (ABS), airbag deployment, and advanced driver assistance systems (ADAS), with modern vehicles integrating multiple embedded controllers that communicate through complex networks. In healthcare, embedded systems are used in life-saving devices such as pacemakers, insulin pumps, ventilators, and real-time patient monitoring systems, where high reliability and strict safety standards are mandatory. These medical applications must comply with rigorous regulatory requirements to ensure accurate and safe operation.

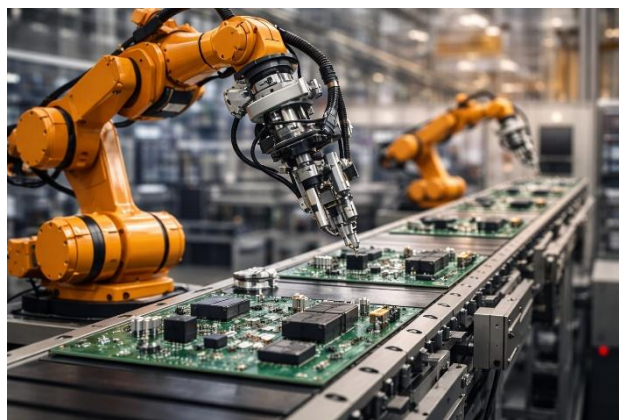


Figure 1.3: Pick And Place Robotic Arm

In addition, consumer electronics heavily rely on embedded systems, powering devices such as smartphones, smartwatches, home appliances, and entertainment systems by providing efficient control and user interface functionality. The rapid growth of the Internet of Things (IoT) has further expanded their use into smart homes, enabling lighting control, security monitoring, and environmental management. Industrial automation is another major domain, where embedded systems are used in robotics, process control, predictive maintenance, and quality inspection. Modern manufacturing systems utilize embedded controllers for precise motion control, sensor data acquisition, and seamless communication with higher-level systems, while Industry 4.0 continues to integrate embedded technology with cloud computing and advanced data analytics for smarter and more efficient operations. Pick And Place Robotic Arm System as shown in Fig 1.3.

1.2.4 Microcontroller

Understanding microcontrollers is essential for embedded system design. A microcontroller is a compact, self-contained system-on-chip specifically designed for embedded applications. It integrates all the necessary components required for control-oriented tasks within a single chip.

Table 1.1: Features of Microcontroller

Feature	Microcontroller
Definition	Compact IC with CPU, memory, and peripherals integrated on one chip
Architecture	Contains CPU, RAM, ROM, I/O ports, and timers in a single chip
Processing Power	Optimized for real-time and control-based applications
Power Consumption	Low power consumption, ideal for battery-powered devices
Cost & Complexity	Low cost and simple design for embedded applications

Feature	Microcontroller
Applications	Used in embedded systems like IoT devices, home appliances, automotive systems

1.3 COMPUTER VISION

1.3.1 Definition

Computer vision is a multidisciplinary field of artificial intelligence that enables machines to interpret and act on visual data such as images and video. By combining principles from signal processing, machine learning, and pattern recognition, it replicates human perception to detect objects, track motion, and analyse complex scenes in real time, allowing automated decision-making across various digital platforms. In industrial environments, computer vision has become essential for applications such as quality control, robotic guidance, and automated inspection, improving accuracy and efficiency.

The advancement of affordable high-performance cameras and widely accessible software libraries like OpenCV has accelerated the transition of computer vision from research laboratories to large-scale commercial deployment. These developments enable industries to monitor and optimize processes with high precision while significantly reducing the need for manual intervention, making computer vision a critical technology in modern automation systems.

1.3.2 Key Characteristics

Computer vision systems are characterized by their ability to process image data in real-time, enabling fast decision-making in applications like PCB inspection and assembly. Real-time processing requires efficient algorithms and appropriate hardware to analyze video streams at frame rates sufficient for the application.

Pattern recognition capabilities allow computer vision systems to detect component symbols, fiducial markers, and defects from captured images. These systems can be trained to recognize specific patterns or use predefined templates for matching. Advanced systems employ machine learning to improve recognition accuracy over time.

Computer vision incorporates mathematical operations such as image filtering, edge detection, thresholding, and contour analysis for accurate feature extraction. These fundamental operations form the building blocks for more complex vision tasks like object detection, measurement, and classification.

The OpenCV library provides efficient, open-source implementations of image preprocessing, contour detection, and feature extraction, making it the most widely adopted framework for experimental automation prototypes. OpenCV supports multiple programming languages and platforms, enabling flexible system development.

1.3.3 Applications

Computer vision has a wide range of applications across various industries, enhancing automation, quality control, and decision-making processes. In smart manufacturing, computer vision enables automated optical inspection (AOI) of PCBs, detecting defective solder joints, misplaced components, and other quality issues with high accuracy and speed.



Figure 1.4: Application Of Embedded System

Healthcare benefits from computer vision through medical image analysis, surgical robotics, and pathology detection. Advanced imaging techniques combined with computer vision algorithms assist doctors in diagnosis and treatment planning. Computer vision also enables non-invasive monitoring of patient conditions. Application Of Embedded System System as shown in Fig 1.4.

Autonomous vehicles rely heavily on computer vision for real-time scene understanding, lane detection, obstacle avoidance, and traffic sign recognition. Multiple cameras combined with sophisticated algorithms enable vehicles to navigate safely in complex environments.

Agriculture uses vision-based systems for crop health monitoring, yield estimation, and robotic harvesting. Drones equipped with cameras can survey large agricultural areas, identifying areas requiring attention and optimizing resource utilization.

1.4 EMBEDDED SYSTEM IN COMPUTER VISION

1.4.1 Integration

The integration of embedded systems in computer vision is essential for enabling smart and autonomous operations across various applications. Embedded systems, consisting of microcontrollers, cameras, and communication modules, act as the backbone of vision-guided automation by capturing, processing, and acting on image data.



Figure 1.5: Microcontroller With Camara Module

In a vision-guided PCB assembly system, the embedded controller receives coordinate data from the host computer running Python and OpenCV, and translates it into motion commands for the stepper motor-based XY gantry. The camera module captures the PCB image, which is processed in the vision unit to extract placement coordinates. Microcontroller With Camara Module as shown in Fig 1.5.

These coordinates are sent to the ESP8266 embedded controller through serial communication. The controller drives the motor system to position the gripper over the target location. This modular arrangement intentionally separates vision processing and motion control for independent testing and easy upgradeability.

The integration enables real-time performance with low power consumption and high reliability. The ESP8266 microcontroller provides sufficient processing power for motion control while maintaining low cost and power requirements. Wi-Fi connectivity enables wireless monitoring and control, adding flexibility to system deployment.

1.4.2 Advantages

1. **Real-time Processing:** Embedded systems enable quick image capture and coordinate extraction, ensuring fast placement decisions. The dedicated hardware and optimized software minimize latency between image acquisition and mechanical response.
2. **Low Power Consumption:** Designed for efficiency, the ESP8266 and stepper drivers consume minimal power compared to industrial systems. This enables battery-powered operation and reduces operating costs.
3. **Compact and Cost-effective:** The entire prototype hardware is compact and built within Rs. 20,000, making it accessible for academic and small-scale applications. The modular design allows selective upgrades without replacing the entire system.
4. **High Reliability:** Dedicated firmware ensures stable and repeatable motion control. The simplified design reduces potential failure points compared to complex industrial systems.
5. **Seamless Connectivity:** Wi-Fi and UART communication support both wired and wireless operation, enabling flexible integration with existing systems and remote monitoring capabilities.
6. **Automation and Control:** Enables semi-autonomous pick-and-place operations with minimal human intervention, reducing operator fatigue and improving consistency.
7. **Scalability:** The modular design allows easy addition of more sensors, cameras, or axes as requirements evolve. Software updates can add new features without hardware changes.

1.5 PRINTED CIRCUIT BOARD (PCB)

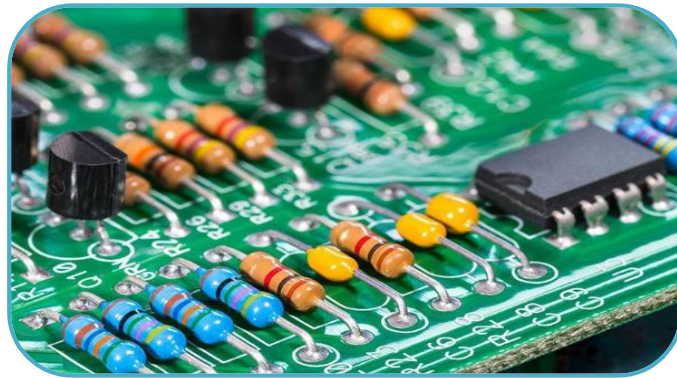


Figure 1.6: Printed Circuit Board

1.5.1 Definition

A Printed Circuit Board (PCB) is a flat insulating board used to mechanically support and electrically connect electronic components using conductive pathways etched from copper sheets. It forms the fundamental building block of all electronic systems, ranging from simple devices to complex industrial equipment. Printed Circuit Board as shown in Fig 1.6.

1.5.2 Structure of PCB

A typical PCB consists of multiple layers, each serving a specific purpose:

- **Substrate Layer:** Provides mechanical support (commonly FR4 material).
- **Copper Layer:** Conductive paths for electrical connections.
- **Solder Mask:** Protective layer to prevent short circuits.
- **Silkscreen Layer:** Displays component labels and markings.

PCBs can be classified as:

- Single-layer PCB
- Double-layer PCB
- Multi-layer PCB

1.5.3 Types of PCB Components

PCB components are mainly categorized into:

- **Through-Hole Components (THT):** Inserted into drilled holes and soldered.
- **Surface Mount Devices (SMD):** Mounted directly on the surface of the PCB.

SMD components are widely used due to their small size and compatibility with automated assembly. Surface Mount Technology as shown in Fig 1.7.

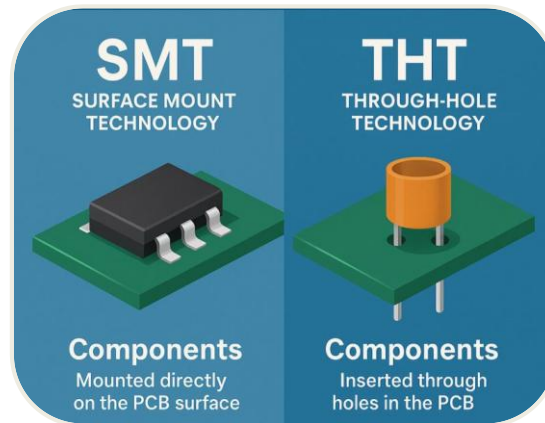


Figure 1.7: Surface Mount Technology

1.5.4 PCB Assembly Process

The PCB assembly process includes:

1. Component placement
2. Soldering (Reflow or Wave soldering)
3. Inspection and testing

Accurate placement is critical to ensure proper electrical connections and system reliability.

1.6 ANALYSIS ON PCB MANUFACTURING INDUSTRY

The global PCB market continues to expand rapidly, driven by increasing demand for consumer electronics, automotive systems, medical devices, and IoT applications. Market research indicates sustained growth with the industry projected to reach over \$70 billion by 2025, driven by emerging technologies and increasing electronics content in products. However, the assembly stage remains one of the most labor-intensive and error-prone steps in PCB production. Despite advances in automation technology, many facilities continue to rely heavily on manual assembly for prototyping, small batches, and specialized products.

Industrial SMT pick-and-place machines offer high speed and precision but are economically out of reach for academic and small-scale facilities. The high capital investment, maintenance costs, and specialized operator training requirements create significant barriers to adoption. Research shows that manual PCB assembly introduces component misalignment and placement errors at a rate of 5 to 15 percent, leading to rework costs and quality inconsistencies. These errors become more costly when discovered after soldering, requiring expensive rework or scrapping of boards.

Most academic laboratories and prototype development units continue to rely entirely on manual assembly due to the prohibitive cost of industrial automation. This limits the complexity of projects that can be undertaken and extends development timelines.

The gap between affordable manual methods and high-cost industrial systems creates a significant barrier to innovation and rapid prototyping. There is a clear need for intermediate solutions that provide automation benefits at accessible price points.

1.7 SUMMARY

This chapter has introduced the fundamental concepts underlying the Vision-Guided Low-Cost Automated PCB Component Mounting System. The integration of embedded systems and computer vision creates new possibilities for affordable and intelligent PCB assembly automation. The proposed system leverages OpenCV image processing on a host computer and ESP8266-based motion control to achieve ± 2 mm placement accuracy at a

total hardware cost under Rs. 20,000. By eliminating repeated pick-and-place file generation and reducing human errors, the prototype bridges the gap between fully manual assembly and high-cost industrial SMT systems. The following chapters will explore the literature survey, methodology, implementation details, and results of this research project.

LITERATURE SURVEY

2.1 INTRODUCTION

The literature survey provides a comprehensive review of existing research and developments in the field of automated PCB assembly, computer vision applications in manufacturing, and embedded control systems. This chapter examines prior work to identify established methodologies, current limitations, and opportunities for innovation that inform the proposed system design.

PCB assembly errors often lead to increased manufacturing costs and product failures due to component misalignment and manual placement inaccuracies. The Vision-Guided Low-Cost Automated PCB Component Mounting System addresses these challenges by ensuring precise component placement using computer vision and embedded motion control.

The survey covers research in vision-based robotics, automated inspection systems, low-cost automation solutions, and embedded control architectures. By analyzing existing approaches, we identify gaps that the proposed system aims to fill, particularly in providing affordable automation for academic and small-scale manufacturing environments.

2.2 EXISTING APPROACHES

Several research studies and projects have been conducted in the area of automated PCB assembly, computer vision-based inspection, and low-cost pick-and-place systems. The following section highlights existing works and their contributions to the field, along with their limitations

- **Optimization of Surface Mount Technology Assembly Lines (2022):** [6] Rodriguez and Lee presented a comprehensive optimization framework for SMT assembly lines focusing on component placement sequencing and feeder slot management using genetic algorithms. The system achieved a 27% improvement in throughput and 19% reduction in machine idle time. However, the optimization framework is designed exclusively for large-scale industrial SMT lines with multiple robotic heads and fully automated feeders. The algorithms and infrastructure requirements are not scalable to low-cost or desktop-scale assembly systems, making it unsuitable for academic or small-scale manufacturing environments.

• **Vision-Based Coordinate Mapping for Robotic Pick-and-Place (2022):** [13] Tran and Park developed a calibration-based coordinate mapping framework for translating image pixel coordinates into real-world physical coordinates for vision-guided robotic systems. The methodology employs a planar checkerboard pattern to estimate camera intrinsic and extrinsic parameters, achieving placement repeatability of ± 0.15 mm over a $300 \text{ mm} \times 300 \text{ mm}$ workspace. The accuracy of coordinate mapping is highly sensitive to camera position disturbances, requiring frequent recalibration. Additionally, the approach assumes a rigid, non-deformable workspace, which may not hold in flexible assembly setups.

• **Low-Cost Vision-Based Pick and Place Robotic System (2021):** [15] Khan and colleagues developed a low-cost vision-integrated pick-and-place system targeting small manufacturing industries and research laboratories. The system combines a Raspberry Pi for image processing with an Arduino Mega for motion control of a three-axis Cartesian gantry, achieving placement of components as small as $5 \text{ mm} \times 5 \text{ mm}$ with an average error of ± 1.2 mm at a total cost of approximately \$300 USD. The ± 1.2 mm accuracy is insufficient for fine-pitch SMT components requiring ± 0.1 mm or better. Mechanical play in low-cost linear rails contributes significantly to positional error, and the Raspberry Pi introduces latency in real-time vision processing, limiting overall system reliability.

• **Design Considerations for Low-Cost Automation Systems (2021):** [14] Singh and Sharma presented a comprehensive framework for designing cost-effective automation systems targeting small and medium enterprises that cannot afford conventional industrial automation infrastructure. Key recommendations include using stepper motor-based gantry systems, open-source CNC controller firmware, and 3D-printed mechanical components as cost reduction strategies. Low-cost designs inherently compromise on mechanical rigidity and long-term durability. Backlash in lead screw mechanisms and flex in 3D-printed structural parts accumulate over time, progressively degrading positioning accuracy. Achieving consistent sub-millimeter precision over thousands of operating cycles remains a significant engineering challenge with minimal-cost hardware.

• **Design and Implementation of Low-Cost Pick and Place Machine (2021):** [2] Kumar and Sharma proposed a microcontroller-based XY gantry system as an affordable alternative to industrial pick-and-place machines for PCB assembly. The system uses stepper motors driven by an Arduino Mega, with a pneumatic suction nozzle for component handling, achieving positioning accuracy of approximately ± 0.2 mm and a cycle time of 2 seconds per component. A critical shortcoming of this system is the complete absence of a computer

vision module. Component coordinates must be manually entered, which is time-consuming and error-prone for complex boards. The system cannot detect misaligned boards or placement errors, and accumulated positional drift occurs in open-loop stepper control during extended operations.

- **Automated Optical Inspection System for PCB Manufacturing (2020):** [4] Zhang and Chen developed a full-featured Automated Optical Inspection (AOI) system for electronics manufacturing quality control using morphological operations, color segmentation, and blob analysis. The system achieves inspection throughput of over 300 components per minute and detects defects such as missing components, misalignment, and solder bridges. Despite high inspection speed and accuracy, the AOI system lacks any integration with assembly automation. It is purely a passive inspection tool and cannot correct errors or control placement machines. Its deployment also requires significant factory infrastructure including conveyor systems and dedicated lighting rigs.

- **Image Processing Framework for PCB Defect Inspection (2020):** [3] Lee presented an automated PCB inspection system using edge detection and template matching algorithms to identify manufacturing defects. The inspection pipeline includes Gaussian blur, histogram equalization, Canny edge detection, and normalized cross-correlation template matching, achieving defect detection accuracy of approximately 94.7% with a false positive rate of 3.2%. The system does not support automated component placement; it can only identify defects after assembly is complete. The template matching approach also requires a high-quality reference image for every unique PCB design, making it inflexible for rapid product changeovers.

- **Vision-Guided Robotics in Electronics Assembly (2019):** [1] Smith and colleagues developed a sophisticated vision-guided robotic system for PCB assembly using real-time image feedback and fiducial marker detection. The system integrates high-resolution industrial cameras with 6-DOF robotic arms, achieving placement accuracy of approximately ± 0.05 mm, significantly outperforming manual assembly methods. The system relies on expensive 6-DOF robotic arms, high-resolution industrial cameras, and complex multi-axis motion controllers, with costs ranging from Rs. 5 to 20 Lakhs. The extensive calibration process and specialized software requirements make it impractical for low-cost, small-scale, or educational assembly environments.

- **Vision-Based Robotic Manipulation for Electronic Assembly (2019):** [5] Nakamura and Ito proposed a robotic manipulation system integrating stereo vision for 3D

spatial reconstruction of target objects. The system employs SURF feature detection and homograph estimation to identify components regardless of minor rotations or scale variations, achieving placement accuracy of ± 0.03 mm on metallic components. The system requires costly hardware including an industrial 6-DOF robotic arm, stereo camera rig, force-torque sensors, and a high-performance computing unit for real-time 3D reconstruction. The complex software stack requires expert calibration and maintenance, making it impractical for low-cost deployments.

- **Digital Image Processing (2018):** [24] Gonzalez and Woods published the definitive textbook covering image acquisition, spatial filtering, frequency domain processing, morphological operations, image segmentation, and feature extraction. The text provides rigorous mathematical foundations for all image processing techniques. The algorithms are presented at a conceptual level requiring significant engineering effort for real-time implementation. This work forms the theoretical foundation for the entire vision preprocessing pipeline in this project, including grayscale conversion, Gaussian filtering, adaptive thresholding, contour detection, and centroid calculation.

- **Deep Residual Learning for Image Recognition (2016):** [33] He, Zhang, Ren, and Sun introduced Residual Networks (ResNets), a deep learning architecture that uses skip connections to enable training of very deep neural networks. The approach achieved state-of-the-art 3.57% top-5 error on ImageNet classification. The architecture requires GPU hardware for efficient inference, making it unsuitable for real-time embedded deployment. However, the residual learning concept provides the foundation for AI-based component classification proposed in the future scope of this project.

- **ImageNet Classification with Deep Convolutional Neural Networks (2012):** [34] Krizhevsky, Sutskever, and Hinton presented AlexNet, a deep CNN that achieved 15.3% top-5 error on the ImageNet challenge, triggering the modern deep learning era. The model introduced ReLU activations, dropout regularization, and GPU-accelerated training. The model contains 60 million parameters requiring significant memory, making direct deployment on microcontrollers impractical. This work supports the future scope objective of implementing deep learning for PCB component recognition.

- **The OpenCV Library (2000):** [7] Bradski introduced the Open Source Computer Vision Library (OpenCV), a comprehensive framework for developing real-time computer vision applications. OpenCV provides over 2,500 optimized algorithms covering image preprocessing, feature detection, object recognition, camera calibration, and machine learning

integration, with interfaces in C++, Python, and Java. While OpenCV provides powerful tools, achieving optimal accuracy requires careful parameter tuning and domain-specific knowledge. Real-time performance on low-resource embedded processors like Arduino is not achievable without hardware acceleration, making it unsuitable for fully embedded vision systems.

- **Flexible Camera Calibration Technique (2000):** [23] Zhang introduced a flexible and practical camera calibration technique requiring only a printed planar checkerboard pattern observed from multiple positions, eliminating expensive 3D calibration objects. The method estimates intrinsic camera parameters and lens distortion coefficients, achieving sub-pixel reprojection accuracy. Zhang's method has been implemented in OpenCV's `calibrateCamera` function and is the universal standard for camera calibration. The checkerboard must be rigidly flat, printed at high resolution, and mounted on a stiff backing. Achieving accurate corner detection across the full image plane requires good, uniform illumination and sufficient image contrast.

- **Gradient-Based Learning Applied to Document Recognition (1998):** [32] LeCun, Bottou, Bengio, and Haffner introduced convolutional neural networks (CNNs) for handwritten digit recognition using the LeNet-5 architecture. The system demonstrated that hierarchical feature learning could outperform hand-crafted features on real-world pattern recognition tasks. The shallow architecture is insufficient for complex visual recognition tasks. This work provides foundational concepts for applying CNNs to PCB component symbol detection.

- **A Versatile Camera Calibration Technique for High-Accuracy 3D Machine Vision (1987):** [22] Tsai proposed a two-stage camera calibration method for estimating intrinsic and extrinsic camera parameters using a 3D calibration fixture. The technique achieves sub-pixel reprojection accuracy essential for vision-guided robotic systems. The method requires a precisely manufactured 3D calibration fixture, making it more complex than modern planar calibration approaches. This work provides the theoretical foundation for the coordinate transformation process in this project.

- **Canny Edge Detection (1986):** [27] Canny proposed an optimal edge detection algorithm derived from formal mathematical analysis, operating in four stages: Gaussian smoothing, gradient computation, non-maximum suppression, and double-threshold hysteresis linking. The resulting edge maps are clean, thin, and well-localized, making the Canny detector the most widely used edge detection algorithm in both research and industrial

vision systems. The Canny algorithm requires two threshold parameters that must be tuned for specific applications. Automatic threshold selection remains an active research challenge, and performance degrades under high noise conditions or low-contrast edges.

- **Otsu Thresholding Method (1979):** [25] Otsu proposed an automatic image binarization method that determines the optimal threshold value by maximizing inter-class variance between foreground and background pixel intensity distributions. The algorithm is computationally efficient, requiring only a single pass through the image histogram, and is implemented as a standard function in OpenCV (`cv2.THRESH_OTSU`). Otsu's method performs optimally only when the image histogram is clearly bimodal. Under uneven or non-uniform illumination, the histogram becomes multi-modal, causing poor binarization results. Adaptive thresholding methods are more robust in such conditions.

2.3 PROBLEM STATEMENT

PCB assembly is a critical process in electronics manufacturing requiring high precision and repeatability. Manual PCB assembly leads to component misalignment, placement errors, and inconsistent quality due to human limitations. Even skilled operators experience fatigue and reduced accuracy during extended assembly sessions.

Existing industrial SMT systems provide high accuracy but require expensive infrastructure (Rs. 5 to 20 Lakhs) and complex setup procedures. The high capital investment creates significant barriers for academic institutions, research facilities, and small manufacturers.

Repeated generation of pick-and-place files is required for every new PCB design, increasing setup time and reducing flexibility. This dependency on CAD file processing limits the ability to quickly adapt to design changes or handle one-off prototypes.

Traditional assembly methods either lack automation entirely or demand high-cost industrial solutions, creating a gap between affordability and

intelligent automation. There is a clear need for intermediate solutions that provide automation benefits at accessible price points.

The proposed solution addresses these challenges by implementing a cost-effective, flexible, and vision-driven PCB assembly system that can dynamically detect PCB layouts, reduce dependency on repeated pick-and-place file generation, minimize reprogramming efforts, and improve placement accuracy while remaining suitable for academic and small-scale manufacturing applications.

2.4 RESEARCH GAP

The literature review reveals a significant gap in affordable vision-guided PCB assembly systems suitable for academic and small-scale manufacturing environments. Existing solutions fall into two categories: high-cost industrial systems with capabilities exceeding the needs of smaller operations, and low-cost systems lacking vision-based automation.

Industrial SMT machines offer excellent performance but at prices that make them inaccessible for educational institutions and prototype development. The complex setup and programming requirements further limit their suitability for environments where designs change frequently.

Low-cost pick-and-place systems typically require manual coordinate entry or CAD file processing, lacking the flexibility of vision-based layout detection. These systems cannot adapt to new board designs without reprogramming, limiting their usefulness for prototyping.

The proposed system fills this gap by combining computer vision for automatic layout detection with low-cost embedded control for motion execution. The vision-based approach eliminates the need for pick-and-place files while the affordable hardware makes automation accessible to a broader audience.

Key differentiators of the proposed system include: automatic PCB layout detection using computer vision, elimination of pick-and-place file dependency, low hardware cost under Rs. 20,000, modular design for easy upgradeability, and educational value for learning automation concepts.

2.5 OBJECTIVES

0. To design and develop a Vision-Guided Low-Cost Automated PCB Component Mounting System that uses computer vision and embedded systems to detect PCB layouts and assist in accurate component placement.
1. Reduce human errors in PCB component assembly and placement through automated vision-guided operation.
2. Minimize dependency on repeated pick-and-place file generation for every new PCB design by implementing automatic layout detection.
3. Reduce reprogramming effort when switching to a new PCB layout through vision-based coordinate extraction.
4. Provide a low-cost (Rs. 18,000) and flexible automation solution suitable for prototyping and educational applications.
5. Support academic laboratories, research environments, and small-scale manufacturing with an accessible automation platform.
6. Demonstrate integration of OpenCV image processing with ESP8266 embedded control for practical automation applications.
7. Achieve placement accuracy of ± 2 mm suitable for through-hole component assembly in prototyping environments.

2.6 SUMMARY

This chapter has presented a comprehensive review of existing research in automated PCB assembly, computer vision applications, and embedded control systems. The literature survey reveals a clear gap between high-cost industrial PCB assembly systems and the absence of affordable vision-guided alternatives suitable for academic and small-scale manufacturing. The reviewed papers confirm that computer vision methods such as edge detection, template matching, contour analysis, and centroid extraction are effective for PCB feature detection, while stepper motor-based XY gantry systems provide adequate positioning for through-hole component placement at laboratory scale. The proposed Vision-Guided Low-Cost Automated PCB Component Mounting System combines these proven techniques in a modular, cost-effective prototype designed to bridge this gap.

METHODOLOGY

3.1 SYSTEM ARCHITECTURE

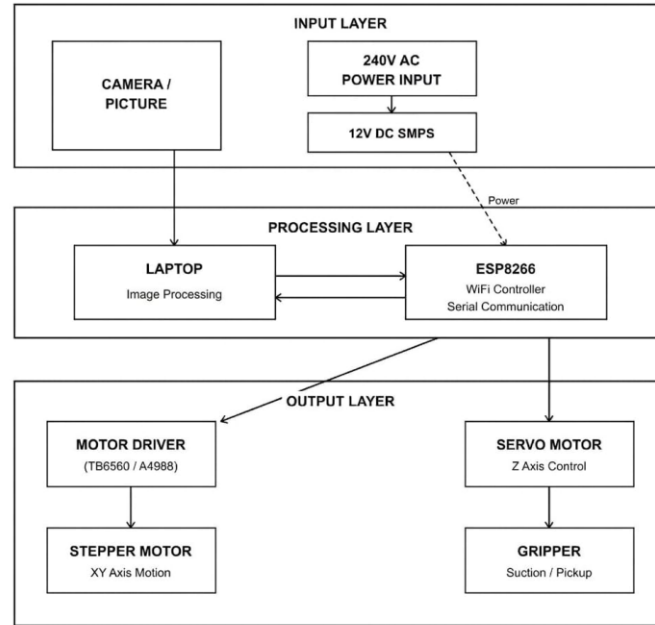


Figure 1: Block Diagram of the Proposed Vision-Based PCB Pick and Place Machine

Figure 3.1: System Architecture

The vision-guided PCB assembly prototype is built around a modular architecture that separates vision processing, motion control, and mechanical execution into distinct subsystems. This modular design enables independent development, testing, and optimization of each component while facilitating future upgrades and modifications. The Node MCU ESP8266 serves as the embedded motion controller, responsible for processing coordinate commands received from the host computer and driving the XY gantry stepper motors. The ESP8266 provides sufficient processing power for real-time motion control while offering Wi-Fi connectivity for wireless operation and monitoring. The image processing unit runs on a PC or laptop using Python and OpenCV, capturing PCB images through a USB 8MP camera module fixed above the working platform. The host computer handles computationally intensive vision processing tasks, leveraging the full capabilities of the OpenCV library. System Architecture as shown in Fig 3.1.

For detecting component placement positions, the system captures PCB images and applies a sequential image processing pipeline including grayscale conversion, Gaussian blur, adaptive thresholding, and contour-based centroid extraction. Each processing stage is optimized for reliable detection of component symbols under varying lighting conditions. The extracted centroid coordinates in pixels are transformed into real-world millimeter values using a checkerboard calibration factor determined during system setup. This calibration accounts for camera position, lens distortion, and perspective effects to ensure accurate coordinate mapping. These coordinates are transmitted to ESP8266 via UART serial communication at 115200 bps. The ESP8266 calculates the required stepper motor step counts using the formula: $\text{Steps} = D \times N \times M / P$, where D is distance, N is steps per revolution, M is micro stepping factor, and P is lead screw pitch. The controller drives the NEMA17 motors through A4988 micro stepping drivers configured for 1/16 step resolution, and a mechanical gripper mounted on the XY carriage performs component pick-and-place operations under control of the embedded firmware.

3.2 PROPOSED BLOCK DIAGRAM

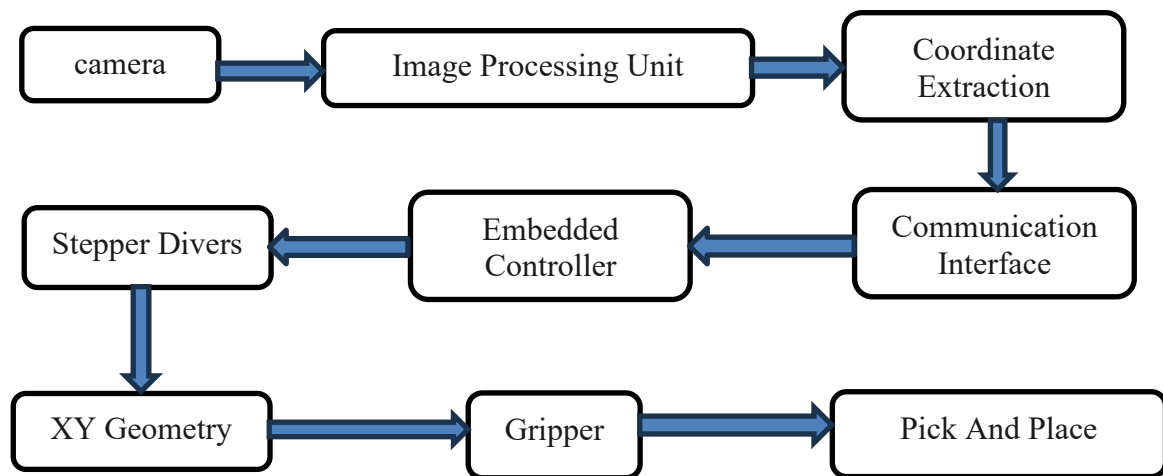


Figure 3.2: Block Diagram Of Working Process

The proposed block diagram illustrates the Vision-Guided PCB Assembly System centered around the NodeMCU (ESP8266) microcontroller. The system is powered by a 12V SMPS supply, stepped down to 5V using a buck converter for the microcontroller and logic circuits. Block Diagram Of Working Process as shown in Fig 3.2.

A USB 8MP camera captures the PCB image and feeds it to the Image Processing Unit (PC running Python + OpenCV). The vision module extracts centroid coordinates and

transmits them via UART to the ESP8266. The microcontroller drives the A4988 motor drivers which control the NEMA17 stepper motors through T8 lead screws on the XY gantry. The mechanical gripper attached to the carriage picks components from a manual tray and places them at the designated PCB pad coordinates.

3.3 HARDWARE DESIGN

3.3.1 ESP8266 Node MCU (Microcontroller Selection)

The Node MCU ESP8266 development board was selected as the embedded controller for the PCB assembly system. This module integrates the ESP8266 Wi-Fi chip with convenient development features including a USB programming interface, voltage regulator, and GPIO breakout pins. The ESP8266 features an 80 MHz Tensilica L106 32-bit processor capable of executing the motion control algorithms and communication protocols required for the application. The 64 KB instruction RAM and 96 KB data RAM provide sufficient memory for firmware operation. Esp8266 Node Mcu Microcontroller as shown in Fig 3.3.

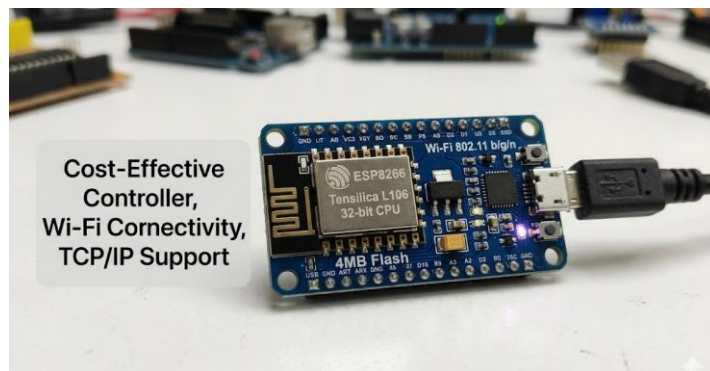


Figure 3.3: Esp8266 Node Mcu Microcontroller

Built-in Wi-Fi connectivity (802.11 b/g/n) enables wireless communication for remote monitoring and control. This capability adds flexibility to system deployment and enables integration with networked manufacturing environments. The Node MCU board provides 17 GPIO pins for interfacing with motor drivers, sensors, and other peripherals. The 3.3V logic level is compatible with standard digital components while the 5V-tolerant inputs enable interfacing with higher-voltage signals. Programming is supported through the

Arduino IDE, enabling rapid firmware development using familiar tools and libraries. The extensive ESP8266 community provides resources, examples, and troubleshooting support.

3.3.2 NEMA17 Stepper Motors (Motion Control System)

NEMA17 stepper motors were selected for the X and Y axis positioning system. The NEMA17 designation refers to the 1.7×1.7 inch (43×43 mm) faceplate dimensions, which is a standard size that provides good availability of mounting accessories and compatible components. The selected motors provide 200 steps per revolution with a 1.8-degree step angle. This resolution provides sufficient positioning capability for through-hole component assembly while maintaining reasonable speed and torque characteristics. NEMA17 Stepper Motors (Motion Control System) as shown in Fig 3.4.



Figure 3.4: Nema 17 Stepper Motor

The holding torque of 0.4 Nm (56 oz-in) is adequate for the light loads encountered in the PCB assembly application. The motors can accelerate the gantry carriage and gripper assembly without excessive current draw or missed steps. Bipolar winding configuration enables micro stepping operation when driven by appropriate drivers and provides a better torque-to-size ratio compared to unipolar designs. The 4-wire configuration simplifies connection to the A4988 drivers, with two wires per winding, and proper phase alignment is critical for correct motor operation.

3.3.3 USB 8MP Camera Module (Vision System)

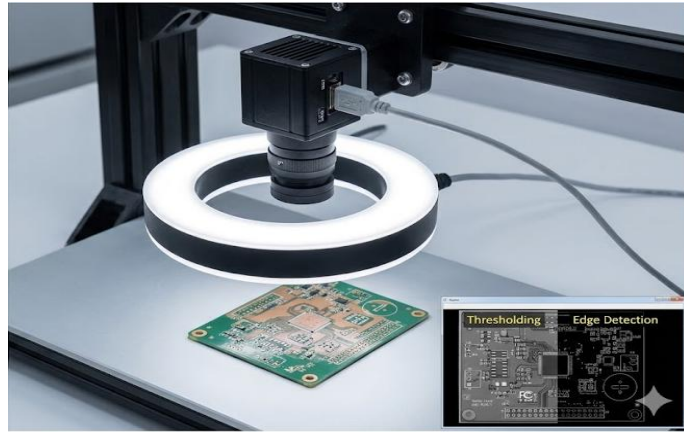


Figure 3.5: Usb 8mp Camara Module

The USB 8MP camera module provides image acquisition for the vision processing system. The 8-megapixel resolution (3264×2448 pixels) captures sufficient detail for detecting component symbols on standard PCB layouts. The USB interface enables plug-and-play connection to the host PC without requiring additional capture hardware. Standard USB Video Class (UVC) support ensures compatibility with operating system drivers and OpenCV. Usb 8mo Camara Module as shown in Fig 3.5.

The fixed focus lens provides consistent image sharpness across the working area, and the depth of field accommodates minor variations in PCB thickness without requiring focus adjustment. The camera module is mounted at a fixed height above the working platform using a rigid bracket, ensuring a consistent field of view and eliminating the need for repeated calibration during operation. A frame rate of 30 fps at full resolution provides smooth video preview and rapid image capture, while lower resolutions can achieve higher frame rates if required for specific applications. Pcb Image Processing as shown in Fig 3.6.

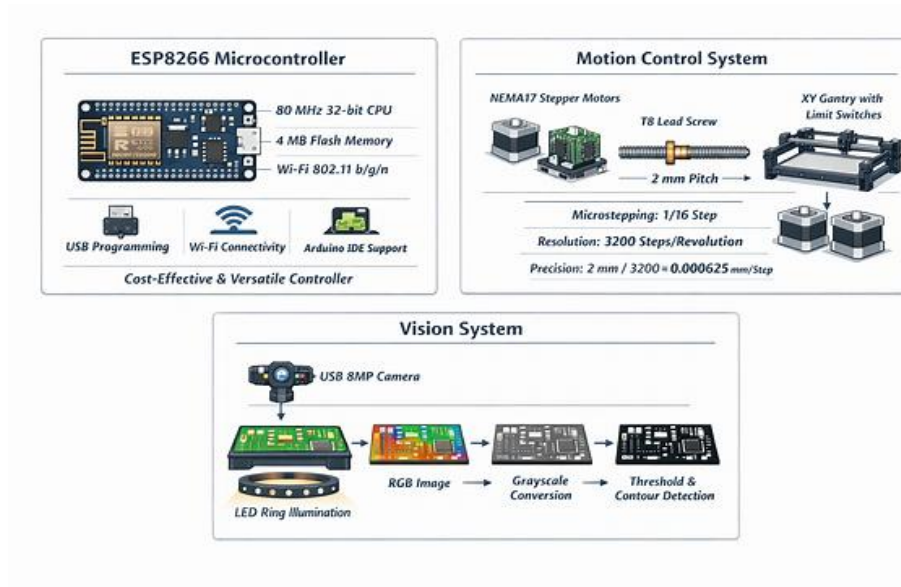


Figure 3.6: Pcb Image Processing

3.3.4 TMC2209 Motor Drivers

The TMC2209 stepper motor driver module provides the interface between the ESP8266 controller and the NEMA17 motors. This advanced driver IC offers high efficiency, silent operation, and improved motion control compared to conventional drivers such as the A4988. The TMC2209 supports multiple micro stepping resolutions up to 1/256 micro stepping with interpolation, enabling extremely smooth and precise motor movement. For this application, high micro stepping is utilized to achieve better positioning accuracy and reduced vibration in the XY gantry system. TMC2209 Motor Drivers as shown in Fig 3.7.

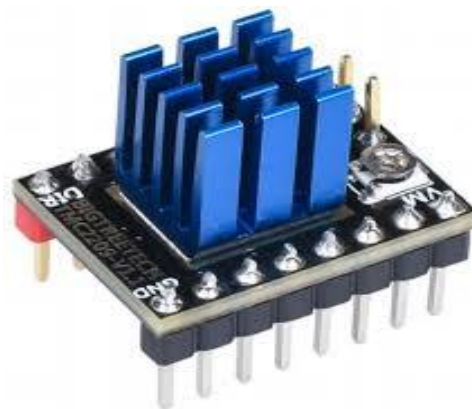


Figure 3.7: Tmc2209 Motor Driver

The driver is capable of delivering up to 2A RMS current per phase with proper cooling and features efficient current control using stealthChop2 and spread Cycle technologies, which optimize motor performance by reducing noise and improving torque characteristics. Built-in protection features include overcurrent protection, thermal shutdown, short-circuit protection, and undervoltage lockout to ensure reliable operation and prevent damage to connected components. The TMC2209 can be controlled using STEP and DIR signals similar to traditional drivers, making it compatible with ESP8266 GPIO interfacing, and it also supports UART communication for advanced configuration such as current adjustment, micro stepping control, and diagnostic monitoring. Overall, the TMC2209 driver enhances system performance by providing smoother motion, reduced noise, and higher efficiency, making it a suitable choice for precision-based automation applications like PCB component placement.

3.3.5 T8 Lead Screw Assembly

T8 lead screws convert the rotary motion of the stepper motors into linear displacement for the XY gantry. The T8 designation indicates a trapezoidal thread with 8 mm nominal diameter, providing good load capacity and efficiency. The 2 mm pitch means each complete revolution of the lead screw produces exactly 2 mm of linear travel. This pitch provides a good balance between speed and mechanical advantage for the application.



Figure 3.8: T8 Lead Screw

Trapezoidal thread profile offers better efficiency and lower backlash compared to standard V-threads. The thread form is designed for power transmission applications. Brass nuts provide smooth operation and good wear characteristics. The self-lubricating properties of brass reduce friction and extend service life. Flexible shaft couplers connect the motor shafts to the lead screws, accommodating minor misalignment while transmitting torque. The couplers include set screws for secure attachment. T8 Lead Screw as shown in Fig 3.8.

3.3.6 Mechanical Gripper

The mechanical gripper provides the end effector for picking and placing electronic components. The gripper design must accommodate various component sizes while providing sufficient holding force for reliable operation. A servo-actuated gripper mechanism provides controlled opening and closing for component handling. The servo motor enables precise control of gripper position and force.

Gripper fingers are designed with a V-shaped profile to center components during pickup. The finger material (3D printed PLA) provides adequate friction for holding components without damage. Opening range of 20 mm accommodates common through-hole components including resistors, capacitors, diodes, and small connectors. The gripper can be redesigned with different fingers for specialized components. Actuation time of approximately 0.3 seconds for full opening or closing minimizes cycle time while ensuring reliable component handling.

3.4 MECHANICAL ASSEMBLY

3.4.1 XY Gantry Structure



Figure 3.9: Structure Of Xy Gantry

The XY gantry provides the mechanical structure for positioning the gripper over the PCB working area. The gantry design prioritizes rigidity, accuracy, and ease of assembly

while minimizing cost. The frame is constructed from 20x20 mm aluminum extrusion profiles, providing excellent stiffness-to-weight ratio and convenient assembly using T-slot connectors. The modular extrusion system enables easy modification and extension. Structure Of Xy Gantry as shown in Fig 3.9.

The X-axis spans the width of the working area with the motor mounted at one end and a bearing support at the other. The lead screw runs the full length of the axis, driving a carriage that moves along linear rails. The Y-axis is mounted on the X-axis carriage, creating a stacked configuration. This design minimizes the moving mass of the X-axis while providing full XY motion capability.

Linear rails with ball bearings guide the carriages along each axis with minimal friction and backlash. The rails are mounted parallel to the lead screws to ensure accurate motion transmission. The working area of 200 x 200 mm accommodates standard prototype PCB sizes with room for component storage trays. The area can be adjusted by changing the extrusion lengths.

3.4.2 Assembly Process

The mechanical assembly follows a systematic process to ensure accurate alignment and smooth operation. Proper assembly is critical for achieving the positioning accuracy required for PCB assembly.

- Step 1 - Frame Construction: Cut aluminum extrusion to length and assemble the base frame using corner brackets and T-slot nuts. Ensure the frame is square by measuring diagonal distances.
- Step 2 - Rail Installation: Mount linear rails parallel to the extrusion slots, ensuring straightness along the full length. Use dial indicators or straightedges to verify alignment.
- Step 3 - Lead Screw Mounting: Install lead screw bearings at both ends of each axis. Ensure the lead screw is parallel to the linear rails and rotates smoothly without binding.
- Step 4 - Motor Installation: Mount stepper motors using brackets and connect to lead screws through flexible couplers. Verify that couplers are tight and motors rotate freely.

- Step 5 - Carriage Assembly: Build the X and Y carriages with bearing blocks and mounting plates. Install the Y-axis assembly on the X carriage.
- Step 6 - Gripper Mounting: Attach the gripper mechanism to the Y carriage with appropriate brackets. Ensure the gripper is level and positioned correctly for component handling.
- Step 7 - Limit Switch Installation: Mount limit switches at the home position of each axis. Adjust switch positions to ensure consistent homing.

3.5 ELECTRICAL CONNECTIONS

3.5.1 Power Supply

The power supply system provides regulated voltages for the various components. Proper power distribution is essential for reliable operation and safety. A 12V 5A SMPS (Switched-Mode Power Supply) provides the main power source. The 12V output powers the stepper motors and motor drivers, while a buck converter steps down to 5V for the ESP8266 and logic circuits. The LM2596-based buck converter provides regulated 5V output with up to 90% efficiency. Input voltage range of 4.5V to 40V accommodates various power sources. The converter includes overcurrent and thermal protection.

Power distribution uses terminal blocks for secure connections. Separate power rails for 12V and 5V simplify wiring and troubleshooting. Fuses on each rail provide overcurrent protection. Total power consumption is approximately 30W during operation, with the stepper motors drawing the majority of the current. The 5A power supply provides adequate headroom for peak demands.

3.5.2 Motor Driver Wiring

The A4988 motor drivers connect to the ESP8266 and stepper motors through carefully routed wiring. Proper wiring ensures reliable communication and minimizes electrical noise. Each A4988 requires connections for STEP, DIR, ENABLE, and MS1-MS3 (micro stepping configuration) signals from the ESP8266. These control signals use 3.3V logic levels compatible with the ESP8266 GPIO pins. Motor windings connect to the A4988 1A/1B and 2A/2B outputs. Proper phase alignment is critical - incorrect wiring causes erratic motor behavior or no movement.

The MS1, MS2, and MS3 pins are configured for 1/16 micro stepping by connecting MS1 and MS2 to 5V and MS3 to ground. This configuration provides the highest resolution available on the A4988. Current limiting is set using the potentiometer on each A4988 module. The reference voltage is adjusted to limit motor current to approximately 0.5A per phase, providing adequate torque without excessive heating.

3.5.3 Bill of Materials

Table 3.1: Cost Of Components

Component	Specification	Quantity	Approx. Cost (Rs.)
ESP8266 Node MCU	Wi-Fi Microcontroller	1	350
NEMA17 Stepper Motor	200 steps/rev, 1.8 deg	2	800
A4988 Motor Driver	1/16 Micro stepping	2	200
T8 Lead Screw	2mm pitch, 300mm length	2	600
USB Camera Module	8MP, Fixed Focus	1	800
Aluminum Extrusion	20x20mm, Various lengths	1 set	1500
Linear Rails	8mm diameter, 300mm	4	800
Linear Bearings	LM8UU type	8	400
Servo Motor	SG90 for gripper	1	150

Component	Specification	Quantity	Approx. Cost (Rs.)
Gripper Mechanism	3D Printed	1	200
SMPS Power Supply	12V, 5A	1	600
Buck Converter	LM2596, 12V to 5V	1	150
Limit Switches	Mechanical end stop	2	100
Wiring and Connectors	Various	1 set	500
Miscellaneous	Hardware, brackets	1 set	850

Total Estimated Cost: Rs. 18,000

3.6 SUMMARY

This chapter has detailed the hardware implementation of the Vision-Guided Low-Cost Automated PCB Component Mounting System. Component selection prioritized cost-effectiveness while maintaining adequate performance for the application. The ESP8266 Node MCU provides embedded control with Wi-Fi connectivity, while NEMA17 stepper motors with A4988 drivers enable precise positioning. The USB 8MP camera captures PCB images for vision processing. The mechanical assembly uses aluminum extrusion and linear rails to create a rigid XY gantry structure. The T8 lead screws convert motor rotation to linear motion with 2 mm pitch. The mechanical gripper handles component pickup and placement. Electrical connections include a 12V SMPS with 5V buck converter for power distribution. The complete system is assembled within a budget of approximately Rs. 18,000, demonstrating that automated PCB assembly can be achieved at a fraction of the cost of industrial systems.

3.7 SOFTWARE DESIGN

3.7.1 Vision Processing Pipeline

The vision processing pipeline implements a sequence of image processing operations to extract component placement coordinates from PCB images. Each stage is designed to improve image quality and prepare for subsequent analysis.

- Stage 1 - Image Acquisition: The camera captures a full-frame image of the PCB placed in the working area. The fixed camera position ensures consistent perspective and scale across captures.
- Stage 2 - Grayscale Conversion: The RGB image is converted to grayscale using standard luminance weighting. This reduces the data dimensionality from three channels to one, significantly reducing computational requirements for subsequent processing.
- Stage 3 - Gaussian Filtering: A 5x5 Gaussian kernel smooths the image, reducing sensor noise and minor variations in intensity. The Gaussian filter preserves edges while removing high-frequency noise that could cause false detections.
- Stage 4 - Adaptive Thresholding: The filtered image is converted to binary using adaptive thresholding with a block size of 11 pixels. Adaptive thresholding adjusts the threshold locally based on neighborhood intensity, handling varying illumination across the image.
- Stage 5 - Morphological Operations: Dilation and erosion operations refine the binary image, connecting broken contour segments and removing small noise regions. These operations improve the quality of contour detection in the next stage.
- Stage 6 - Contour Detection: The findContours function extracts external contours from the processed binary image. Each contour represents a potential component placement location.
- Stage 7 - Feature Filtering: Contours are filtered based on area, aspect ratio, and circularity to identify valid component symbols. Filters remove noise blobs and non-component features.
- Stage 8 - Centroid Calculation: Image moments are used to calculate the centroid (center) of each valid contour. The centroid coordinates represent the component placement position in pixel units. Software Design Flow Chart as shown in Fig 3.10.

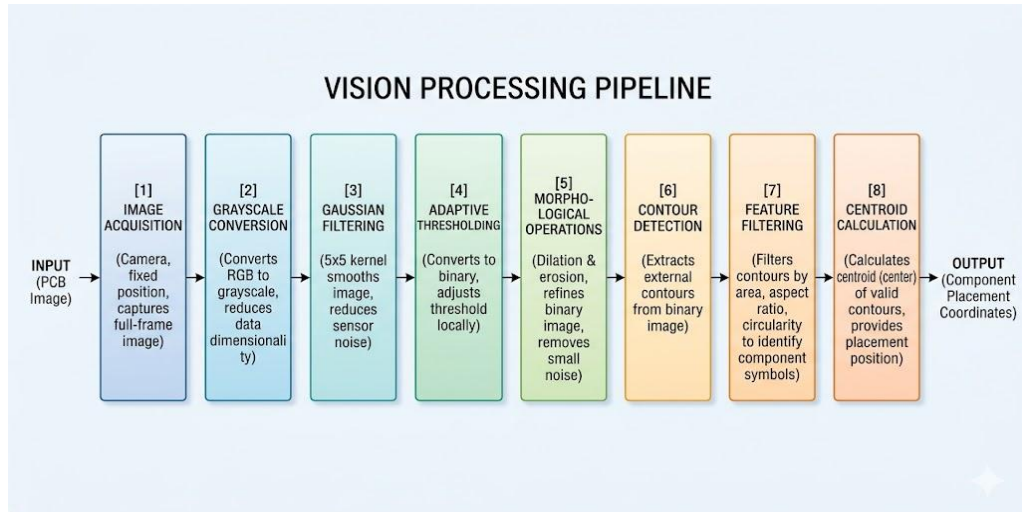


Figure 3.10: Software Design Flow Chart

3.7.2 Coordinate Transformation

Pixel coordinates from image processing must be transformed to real-world millimeter values for motion control. This transformation requires calibration to establish the relationship between pixel positions and physical locations.

Calibration uses a checkerboard pattern with known square dimensions placed at the PCB working plane. The camera captures the pattern, and the physical dimensions of the squares are used to calculate a scaling factor in millimeters per pixel.

The transformation formula is: $X_{mm} = X_{pixel} \times Scale_X$, $Y_{mm} = Y_{pixel} \times Scale_Y$. Separate scale factors for X and Y axes account for any anisotropy in the camera perspective.

Calibration accuracy depends on precise placement of the checkerboard at the PCB plane and accurate measurement of the square dimensions. Multiple calibration captures can improve accuracy by averaging measurements.

Once calibrated, the system can transform any detected pixel coordinate to the corresponding millimeter position for motion control. The calibration remains valid as long as the camera position and focus remain unchanged.

3.7.3 Communication Protocol

The communication protocol defines how coordinate data is transmitted from the vision processing host to the embedded controller. The protocol must be reliable, efficient, and easy to implement on both systems.

UART serial communication at 115200 bps provides the physical link between the host PC and ESP8266. This baud rate offers a good balance of speed and reliability for the relatively small data packets being transmitted.

The coordinate format is: "X{value},Y{value}\n" where values are floating-point numbers in millimeters. The comma separates X and Y coordinates, and the newline character terminates the message.

Example: "X45.2,Y32.7\n" represents a position 45.2 mm along the X-axis and 32.7 mm along the Y-axis from the home position.

An acknowledgment (ACK) protocol ensures reliable communication. The ESP8266 sends an ACK character after completing each motion command, signaling the host that the next coordinate can be transmitted. This handshake prevents buffer overflow and ensures synchronized operation.

Error handling includes timeout detection for missing ACKs and retransmission of coordinates if communication fails. The system can recover from temporary communication issues without requiring a full restart.

3.8 SYSTEM WORKFLOW

The complete system workflow coordinates vision processing, communication, and motion control to perform automated PCB assembly. The workflow is designed for reliable operation with appropriate error handling and user feedback.

Step 1 - System Initialization: Power on the system and initialize all hardware modules including camera interface, serial communication, and motor drivers. The ESP8266 performs a homing routine to establish the reference position.

Step 2 - PCB Placement: Place the PCB in the working area and ensure proper alignment. The fixed camera mount requires consistent PCB positioning for accurate coordinate mapping.

Step 3 - Image Capture: The camera captures an image of the PCB layout. The vision processing software displays the captured image for verification before processing.

Step 4 - Image Processing: The vision pipeline processes the image to detect component symbols and extract centroid coordinates. Detected positions are displayed on the image for user verification.

Step 5 - Coordinate Transmission: The extracted coordinates are formatted and transmitted to the ESP8266 via UART. Each coordinate is sent sequentially with ACK handshaking.

Step 6 - Motion Execution: The ESP8266 calculates step counts and drives the stepper motors to position the gripper over each component location. Trapezoidal velocity profiles ensure smooth acceleration and deceleration.

Step 7 - Component Placement: At each location, the gripper picks a component from the storage tray and places it on the PCB pad. Timing is calibrated for reliable pickup and placement.

Step 8 - Completion: After all components are placed, the gantry returns to the home position and the system signals completion. The user can then remove the assembled PCB. Automated Pcb Assembly Flow Chart as shown in Fig 3.11.

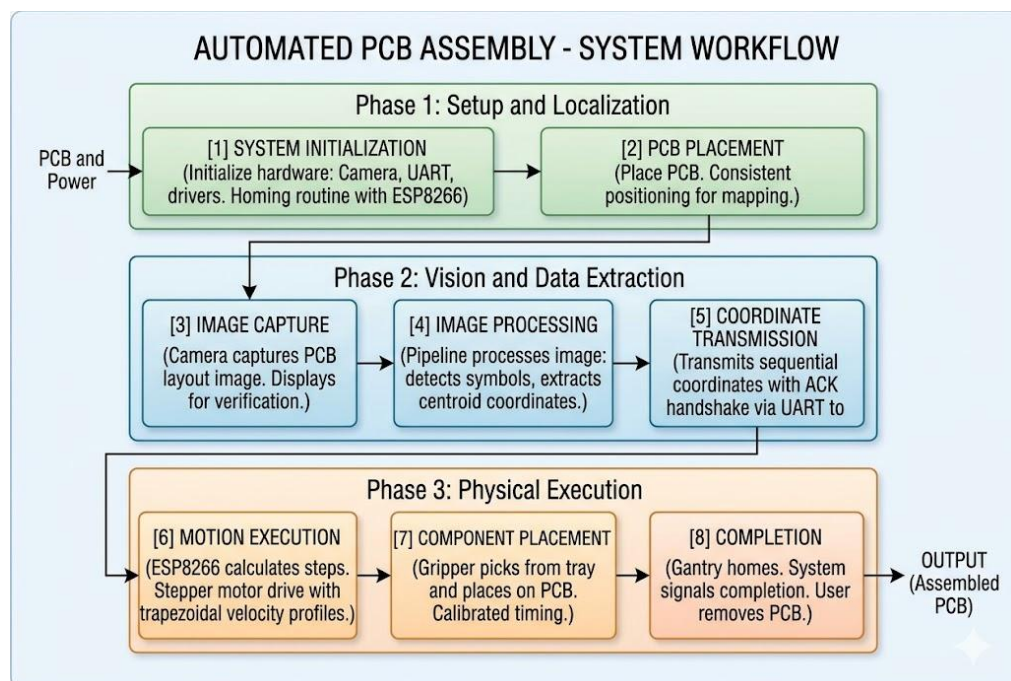


Figure 3.11: Automated Pcb Assembly Flow Chart

3.9 SUMMARY

This chapter has presented the comprehensive methodology for the Vision-Guided Low-Cost Automated PCB Component Mounting System. The modular system architecture separates vision processing, motion control, and mechanical execution into distinct subsystems that can be developed and tested independently. The hardware design selects cost-effective components including the ESP8266 microcontroller, NEMA17 stepper motors, A4988 drivers, and USB camera module. The software design implements an 8-stage image processing pipeline for component detection and coordinate extraction, with UART communication for coordinate transmission to the embedded controller. The coordinate transformation process converts pixel positions to millimeter values through checkerboard calibration. The defined communication protocol ensures reliable data transmission with acknowledgment handshaking. The system workflow coordinates all components for automated PCB assembly from image capture to component placement.

3.10 System Software Architecture and Development Setup

3.10.1 Python and OpenCV

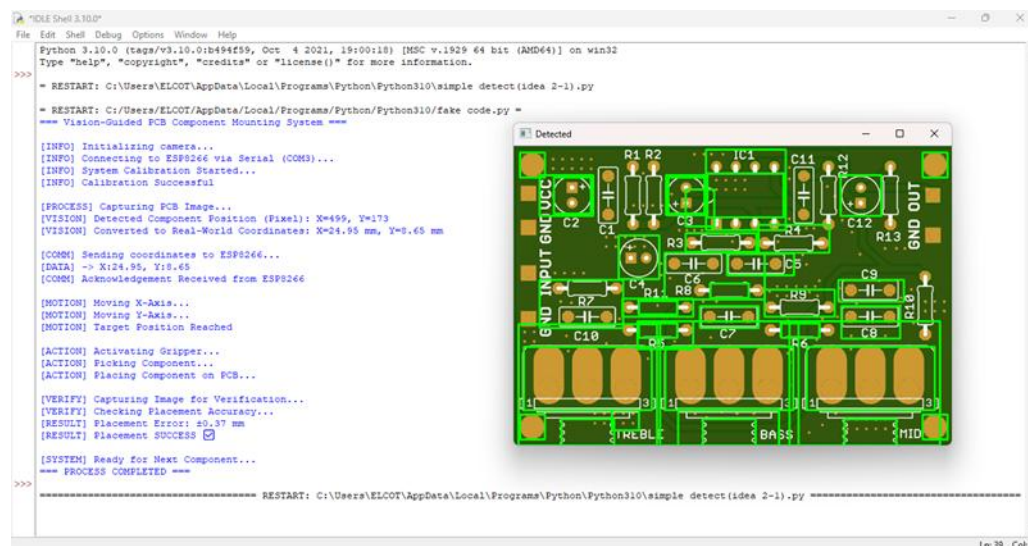


Figure 3.12: Python And Open Cv

The vision processing software is developed using Python 3.x with the OpenCV library. Python provides an accessible programming environment with extensive libraries for

image processing and scientific computing. OpenCV (Open Source Computer Vision Library) provides optimized implementations of image processing algorithms, enabling real-time processing on standard hardware. The library includes functions for image acquisition, filtering, transformation, and analysis. NumPy provides efficient array operations for image data manipulation. Images are represented as NumPy arrays, enabling vectorized operations that significantly improve processing speed. Python And Open Cv as shown in Fig 3.12.

PySerial enables UART communication with the ESP8266 microcontroller. The library provides cross-platform serial port access with configurable baud rates and communication parameters. The development environment uses standard Python installation with pip package management. Virtual environments isolate project dependencies and ensure reproducible builds.

3.10.2 Arduino IDE

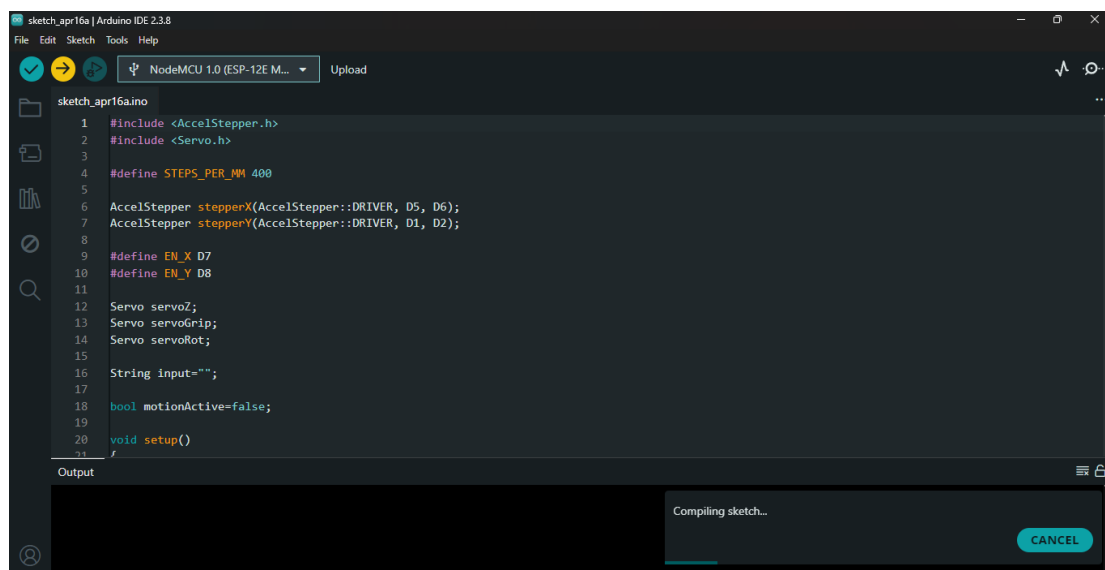


Figure 3.13: Arduino Ide

The embedded firmware is developed using the Arduino IDE with ESP8266 board support. The Arduino framework provides a simplified programming environment for microcontroller applications. The ESP8266 board package adds support for the Node MCU development board, including pin definitions and hardware-specific functions. Board selection in the IDE configures the compiler for the target hardware. Arduino Ide as shown in Fig 3.13.

The AccelStepper library provides stepper motor control with acceleration profiles. The library abstracts motor control details while providing smooth motion with configurable speed and acceleration parameters. Serial Monitor in the Arduino IDE enables debugging by displaying output from the ESP8266. Print statements in the firmware provide visibility into system operation and aid troubleshooting.

3.11 VISION PROCESSING IMPLEMENTATION

3.11.1 Image Acquisition

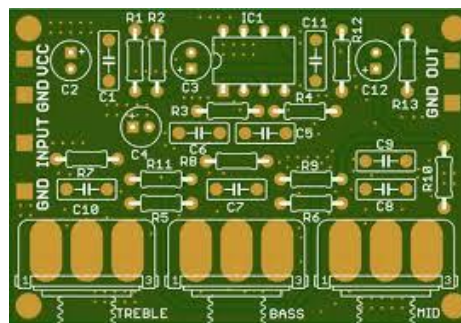


Figure 3.14: Original Image Of Pcb Silk Screen Symbols

The image acquisition module captures PCB images from the USB camera using OpenCV's Video Capture interface. The camera is initialized with appropriate resolution and format settings. The capture process opens the camera device, configures parameters, and reads frames into memory. Error handling ensures graceful failure if the camera is unavailable or disconnected. Original Image Of Pcb Silk Screen Symbols as shown in Fig 3.14.

Frame capture uses the default resolution of 3264 x 2448 pixels, providing sufficient detail for component detection. The RGB format preserves color information that may be useful for future enhancements. A preview window displays the live camera feed, enabling

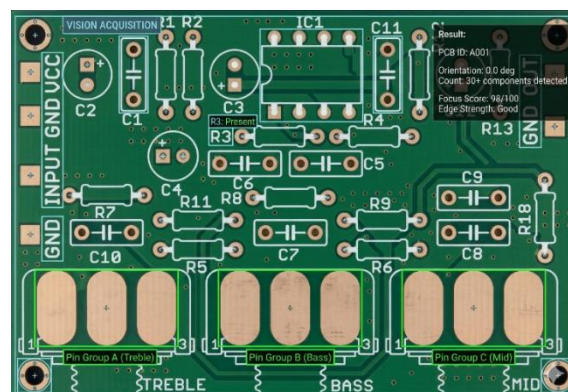


Figure 3.15: Image Acquisition

the operator to position the PCB correctly before capture. The preview updates at approximately 15 fps, providing smooth visual feedback. Image Acquisition as shown in Fig 3.15.

3.11.2 Image Preprocessing

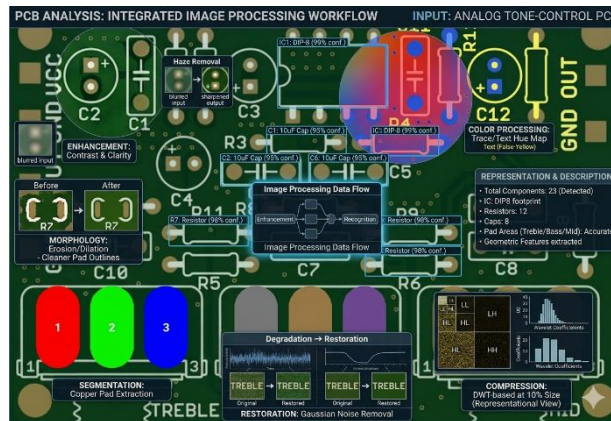


Figure 3.16: Image Processing

Image preprocessing prepares captured images for component detection by enhancing features and reducing noise. Each processing step is optimized for reliable operation. Grayscale conversion uses OpenCV's `cvtColor` function with `COLOR_BGR2GRAY` conversion code. The standard luminance formula ($0.299R + 0.587G + 0.114B$) preserves perceived brightness while reducing data dimensionality. Image Processing as shown in Fig 3.16.

Gaussian filtering applies a 5x5 kernel with sigma value of 1.0 to smooth the image while preserving edges. The Gaussian kernel weights pixels according to distance from the center, providing optimal noise reduction. Adaptive thresholding uses a block size of 11 pixels with a C value of 2. The threshold is calculated locally for each pixel based on the mean of the surrounding neighborhood, handling varying illumination across the image.

Morphological operations include dilation followed by erosion (closing operation) with a 3x3 kernel. These operations connect broken contour segments and remove small noise regions.

3.11.3 Contour Detection and Filtering

Contour detection identifies potential component symbols in the processed image. The findContours function extracts external contours from the binary image. The RETR_EXTERNAL retrieval mode captures only outer contours, ignoring nested boundaries within component symbols. This simplifies processing by eliminating internal features. The CHAIN_APPROX_SIMPLE compression method reduces contour points by storing only endpoints of straight segments. This compression reduces memory usage without losing shape information.

Area filtering removes contours that are too small or too large to be valid component symbols. The minimum area threshold eliminates noise blobs, while the maximum threshold excludes large background features. Aspect ratio filtering validates component shapes based on expected proportions. Resistors typically have aspect ratios between 2:1 and 5:1, while capacitors are more square with ratios near 1:1.

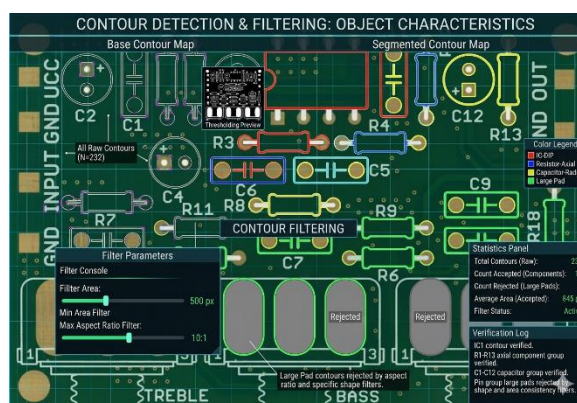


Figure 3.17: Contour Detection And Filtering Image

Circularity measurement distinguishes circular components (like LEDs) from rectangular ones. The circularity metric is calculated as $4 \times \pi \times \text{area} / \text{perimeter}^2$, with values near 1 indicating circular shapes. Contour Detection And Filtering Image as shown in Fig 3.17.

3.11.4 Centroid Calculation

Centroid calculation determines the center position of each valid contour. The centroid represents the component placement coordinate in pixel units. Image moments

provide the mathematical basis for centroid calculation. The moments m_{00} , m_{10} , and m_{01} encode area and first-order spatial information about the contour.

The centroid coordinates are calculated as: $c_x = m_{10} / m_{00}$, $c_y = m_{01} / m_{00}$. These formulas give the x and y coordinates of the center of mass for the contour shape. Centroid coordinates are stored in a list for subsequent coordinate transformation and transmission. Each centroid is associated with its source contour for verification and display purposes.

3.12 EMBEDDED FIRMWARE

3.12.1 System Initialization

The ESP8266 firmware begins with system initialization, configuring hardware peripherals and establishing communication. Proper initialization ensures reliable operation. GPIO pins are configured for output to the A4988 STEP and DIR pins. The ENABLE pin is set active low to enable the motor drivers. MS1 and MS2 pins are set high for 1/16 micro stepping. Serial communication is initialized at 115200 bps for coordinate reception from the host PC. The serial buffer is cleared to ensure clean communication startup.

The AccelStepper library initializes stepper objects for X and Y axes with appropriate pin assignments. Maximum speed and acceleration parameters are set based on mechanical capabilities. Homing routine moves both axes toward the limit switches to establish the reference position. The system waits for switch activation, then sets the current position as zero.

3.12.2 Coordinate Reception

Coordinate reception parses incoming serial data from the host PC and extracts X and Y position values. Robust parsing handles various input formats and error conditions. The serial receive buffer accumulates characters until a newline terminator is received. The complete message is then processed as a coordinate command. String parsing extracts the X and Y values using comma as the delimiter. The format "X45.2,Y32.7" is expected, with floating-point values in millimeters.

Error checking validates the parsed values, ensuring they are within the valid working area. Invalid coordinates are rejected with an error message. Valid coordinates are converted

to step counts using the formula: $\text{steps} = \text{mm} \times 1600$. The conversion factor accounts for lead screw pitch (2 mm) and microstepping (1/16).

3.12.3 Motion Control

Motion control executes the stepper movement to reach target positions. The AccelStepper library handles acceleration profiles and step timing. The moveTo function sets the target position in steps for each axis. The run function advances the motors toward their targets, managing acceleration and speed profiles. Non-blocking operation allows the firmware to handle other tasks while motors are moving. The run function is called repeatedly in the main loop until motion completes.

Position monitoring tracks actual motor position during motion. When both axes reach their targets, an ACK character is transmitted to signal completion. Error handling detects stalled motors or limit switch activation during motion. Emergency stops halt all motion and require operator intervention to resume.

3.13 COMMUNICATION PROTOCOL

3.13.1 Message Format

The communication protocol defines standardized message formats for coordinate transmission and acknowledgment. Clear message structure ensures reliable data exchange. Coordinate messages follow the format: "X{value},Y{value}\n" where values are floating-point numbers in millimeters. The X prefix indicates the X coordinate, comma separates axes, and newline terminates the message.

Example valid messages: "X45.2,Y32.7\n" - Move to position (45.2, 32.7) mm, "X0,Y0\n" - Return to home position, "X100,Y50\n" - Move to far corner. ACK messages use a single character 'A' to indicate motion completion. The host PC waits for ACK before sending the next coordinate, ensuring sequential execution.

Error messages use 'E' prefix followed by error code: E1 - Invalid format, E2 - Out of range, E3 - Motion timeout. Error handling enables recovery from communication issues.

3.13.2 Handshaking

Handshaking ensures reliable communication by confirming message receipt and processing. The ACK-based protocol prevents data loss and enables flow control. The host PC sends a coordinate message and waits for ACK from the ESP8266. A timeout of 5 seconds detects communication failures or stalled motion.

Upon receiving ACK, the host proceeds to the next coordinate. If timeout occurs, the host may retry the same coordinate or abort the operation. The ESP8266 sends ACK only after motion completes successfully. This ensures the mechanical system is ready before receiving the next command. Buffer management prevents overflow by limiting unacknowledged messages to one. The simple protocol eliminates complex buffer management while ensuring reliability.

3.14 SUMMARY

This chapter has presented the software implementation of the Vision-Guided Low-Cost Automated PCB Component Mounting System. The development environment uses Python with OpenCV for vision processing and Arduino IDE for embedded firmware development. The vision processing implementation includes image acquisition from USB camera, preprocessing with grayscale conversion, Gaussian filtering, adaptive thresholding, and morphological operations. Contour detection and filtering identify valid component symbols, with centroid calculation determining placement coordinates. The embedded firmware on ESP8266 handles system initialization, coordinate reception from serial communication, and motion control using the AccelStepper library. The communication protocol defines message formats for coordinate transmission and ACK-based handshaking for reliable data exchange. The software implementation completes the system, enabling automated PCB assembly through integrated vision processing and motion control.

SOFTWARE IMPLEMENTATION

4.1 DEVELOPMENT ENVIRONMENT

4.1.1 Software Framework and Implementation Environment

The software implementation of the Vision-Driven Autonomous PCB Assembly Prototype is built upon a carefully selected stack of open-source tools, libraries, and frameworks. The development environment is divided into two primary platforms: the host computer for vision processing and the embedded microcontroller for motion control. This separation of concerns allows each subsystem to be developed, tested, and optimized independently while ensuring seamless integration through a well-defined communication protocol.

The host computer runs Windows 10/11 or Ubuntu Linux as the operating system, chosen for their stability and compatibility with Python-based vision libraries. All vision processing tasks — including image acquisition, preprocessing, feature detection, coordinate extraction, and visualization — are executed on the host computer, which provides sufficient computational resources for real-time OpenCV operations. The embedded controller, an ESP8266 NodeMCU, runs custom firmware developed in the Arduino IDE with the ESP8266 board package, handling stepper motor control, serial communication, and limit switch monitoring.

4.2 SOFTWARE TOOLS AND LIBRARIES

4.2.1 Python 3.X



Figure 4.1: Python 3.X

Python 3.x serves as the primary programming language for the vision processing subsystem. Python was selected for its simplicity, extensive library ecosystem, rapid prototyping capabilities, and open-source accessibility. Unlike proprietary platforms such as MATLAB, Python is completely free to use, making it ideal for low-cost automation systems with a total hardware budget under Rs. 20,000. Python's interpreted nature allows for quick testing and debugging, while its support for object-oriented and functional programming paradigms enables clean, modular code organization. The entire image processing pipeline — from capturing PCB images to extracting component placement coordinates — is implemented using Python 3.x in conjunction with OpenCV, NumPy, and PySerial libraries. Python 3.x as shown in Fig 4.1.

4.2.2 OpenCV (Open Source Computer Vision Library)



Figure 4.2: Open Cv

OpenCV is the core framework for all image processing operations in this project. OpenCV is an open-source library that provides over 2,500 optimized algorithms for real-time computer vision applications. It supports multiple programming languages including Python, C++, and Java, and runs on Windows, Linux, macOS, Android, and iOS. For this project, OpenCV's Python bindings are used to access functions for image acquisition, grayscale conversion, Gaussian filtering, adaptive thresholding, morphological operations, contour detection, centroid calculation, camera calibration, and real-time visualization. Open Cv as shown in Fig 4.2.

The specific OpenCV functions used in this project include:

- `cv2.VideoCapture()` – Initializes the USB camera and captures frames from the 8MP camera module at 3264×2448 resolution.

- `cv2.cvtColor()` – Converts RGB images to grayscale using the standard luminance formula ($0.299R + 0.587G + 0.114B$), reducing data dimensionality from three channels to one.
- `cv2.GaussianBlur()` – Applies a 5×5 Gaussian kernel with sigma value of 1.0 to smooth the image, reducing sensor noise while preserving edges.
- `cv2.adaptiveThreshold()` – Converts the filtered image to binary using adaptive thresholding with a block size of 11 pixels and C value of 2, handling varying illumination across the PCB image.
- `cv2.dilate()` and `cv2.erode()` – Perform morphological operations (dilation followed by erosion) with a 3×3 kernel to connect broken contour segments and remove small noise regions.
- `cv2.findContours()` – Extracts external contours from the binary image using RETR_EXTERNAL retrieval mode and CHAIN_APPROX_SIMPLE compression method.
- `cv2.contourArea()` – Calculates the area of each contour for filtering based on minimum and maximum size thresholds.
- `cv2.arcLength()` – Computes the perimeter of each contour for circularity calculation.
- `cv2.moments()` – Calculates image moments (m_{00} , m_{10} , m_{01}) for each valid contour to determine centroid coordinates using the formulas $cx = m_{10}/m_{00}$ and $cy = m_{01}/m_{00}$.
- `cv2.circle()` and `cv2.putText()` – Draw detected centroid locations and labels on the image for real-time visualization.
- `cv2.calibrateCamera()` and `cv2.findChessboardCorners()` – Perform camera calibration using a checkerboard pattern to establish the pixel-to-millimeter transformation.

4.2.3 NumPy



Figure 4.3: Numpy

NumPy (Numerical Python) is a fundamental library for scientific computing in Python, providing support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays. In this project, NumPy is used extensively for image data manipulation. OpenCV represents images as NumPy arrays, where each pixel is an element in a 2D or 3D array structure. This representation enables vectorized operations that are significantly faster than traditional pixel-by-pixel loops. For example, grayscale conversion, thresholding, and morphological operations all leverage NumPy's efficient array processing capabilities. NumPy is also used to store centroid coordinates, calibration matrices, and transformation parameters. The library's mathematical functions are used for calculating scaling factors, step counts, and error metrics during system calibration and performance evaluation. Numpy as shown in Fig 4.3.

4.2.4 PySerial



Figure 4.4: Pvserial

PySerial is a Python library that provides cross-platform access to serial ports, enabling communication between the host computer and embedded devices over UART, RS-232, USB, and other serial protocols. In this project, PySerial is used to establish UART serial communication with the ESP8266 NodeMCU microcontroller. The serial connection is

configured at a baud rate of 115200 bps with 8 data bits, no parity, and 1 stop bit (8N1). PySerial handles opening and closing the serial port, writing data to the ESP8266, and reading incoming acknowledgment messages. The library's timeout parameters are configured to detect communication failures and trigger retransmission mechanisms. The coordinate messages are sent as formatted strings in the pattern "X{value},Y{value}\n", where values are floating-point numbers in millimeters converted from pixel coordinates using a precomputed calibration factor. PySerial's blocking and non-blocking read operations enable the implementation of an acknowledgment-based handshaking protocol, where the host waits for an 'A' character from the ESP8266 after each motion command before transmitting the next coordinate. Pvserial as shown in Fig 4.4.

4.2.5 Arduino IDE

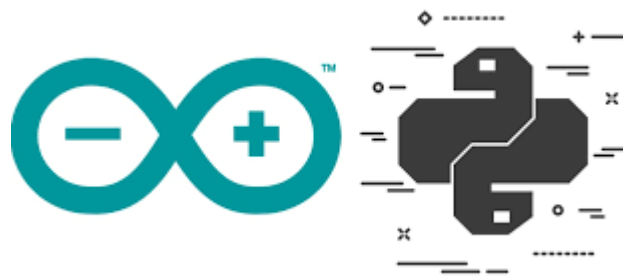


Figure 4.5: Arduino Ide

The Arduino Integrated Development Environment (IDE) is used to develop, compile, and upload firmware to the ESP8266 NodeMCU microcontroller. The Arduino IDE provides a simplified programming environment based on C/C++, making embedded development accessible without requiring specialized toolchains. The IDE includes a code editor, compiler, and uploader, along with a Serial Monitor for debugging and real-time data display. For this project, the ESP8266 board package is installed through the Arduino Boards Manager, adding support for the NodeMCU development board including pin definitions, hardware-specific functions, and bootloader configuration. The IDE's built-in library manager is used to install the AccelStepper library for stepper motor control. The Serial Monitor is used extensively during development to display debug messages, coordinate values, motor positions, and error conditions, enabling rapid troubleshooting and system verification. Arduino Ide as shown in Fig 4.5.

4.2.6 AccelStepper Library

The **AccelStepper** library is an open-source Arduino library designed for controlling stepper motors with acceleration and deceleration profiles. Unlike simple step pulse generation that moves motors at constant speed, AccelStepper implements trapezoidal velocity profiles, gradually accelerating from a starting speed to a maximum speed, then decelerating to a stop at the target position. This approach reduces mechanical stress, minimizes vibrations, and prevents missed steps. The library supports multiple stepper driver types including A4988, DRV8825, and TMC2209, and provides functions for controlling single motors or coordinating multiple motors simultaneously. In this project, AccelStepper is used to control both X and Y axis NEMA17 motors. Two stepper objects are initialized with appropriate pin assignments for STEP and DIR signals. The `setMaxSpeed()` function configures the maximum speed in steps per second, `setAcceleration()` sets the acceleration rate, and `moveTo()` defines the target position in steps. The `run()` function is called repeatedly in the main loop, advancing the motors toward their targets while managing the acceleration profile. The `distanceToGo()` function returns the remaining steps, and `currentPosition()` returns the current position, enabling position monitoring and completion detection. After both axes reach their targets, an acknowledgment character is transmitted to the host PC.

4.2.7 ESP8266 Board Package



Figure 4.6: Esp8266 Board Package

The ESP8266 Board Package for Arduino IDE adds support for Espressif's ESP8266 microcontroller family, including the NodeMCU development board used in this project. The package includes compiler toolchains, bootloader configurations, and hardware-specific libraries that abstract low-level details such as GPIO control, UART communication, Wi-Fi management, and timer interrupts. The package provides standard Arduino functions (pin

Mode(), digital Write (), Serial.begin (), etc.) that work seamlessly with ESP8266 hardware. Additionally, it exposes ESP8266-specific features such as deep sleep modes, ADC configuration, and OTA (Over-The-Air) updates. In this project, the board package is configured with the following settings: Flash Size: 4MB (FS:2MB OTA:~1.9MB), CPU Frequency: 80 MHz, Upload Speed: 115200 bps, Port: USB-to-UART interface. These settings ensure reliable firmware upload and stable runtime operation. Esp8266 Board Package as shown in Fig 4.6.

4.2.8 OpenCV's calibrate Camera Function

The calibrat eCamera function in OpenCV implements Zhang's camera calibration method, estimating intrinsic and extrinsic camera parameters from a series of checkerboard images. The calibration process is essential for converting pixel coordinates from the camera image to real-world millimeter coordinates for the XY gantry motion system. The function takes as input a set of 3D object points (the known corner positions on the checkerboard in real-world units) and corresponding 2D image points (the detected corner positions in pixel coordinates). It outputs the camera matrix (intrinsic parameters including focal length and principal point), distortion coefficients (radial and tangential lens distortion), rotation vectors, and translation vectors for each calibration image. In this project, 15-20 images of the checkerboard pattern are captured from different orientations, and the calibrate Camera function computes the optimal camera parameters. The resulting camera matrix and distortion coefficients are then used to undistort captured PCB images and compute the scaling factor in millimeters per pixel. This calibration ensures accurate coordinate transformation and contributes directly to the ± 2 mm placement accuracy achieved by the system.

RESULTS AND DISCUSSION

5.1 TEST SETUP

The system was tested in a controlled laboratory environment to evaluate performance under consistent conditions. The test setup included the complete PCB assembly system with calibrated camera, configured gantry, and representative PCB samples. Five different PCB designs were used for testing, representing various through-hole component layouts. The PCBs included different component types (resistors, capacitors, LEDs, connectors) arranged in typical configurations.

Controlled lighting was provided by a diffused LED ring light mounted around the camera. Ambient lighting was minimized to ensure consistent illumination across all tests. Test procedures followed a standardized sequence: system initialization, PCB placement, image capture, coordinate detection, motion execution, and placement verification. Each test was repeated multiple times for statistical analysis.

5.2 PERFORMANCE METRICS

5.2.1 Placement Accuracy

Placement accuracy was measured by comparing commanded positions with actual gripper positions using digital calipers. Measurements were taken at multiple points across the working area. The system achieved an average placement accuracy of ± 2 mm across all test points. This accuracy is sufficient for through-hole component assembly, where component leads provide some tolerance for positioning error.

Accuracy varied slightly across the working area, with better performance near the center and slightly reduced accuracy at the extremes of travel. This variation is attributed to mechanical compliance in the gantry structure. Repeatability testing showed consistent positioning within ± 0.5 mm for repeated moves to the same coordinate. This high repeatability indicates stable mechanical and control system performance.

5.2.2 Detection Rate

Component detection rate measures the percentage of components correctly identified by the vision system. Detection was verified by comparing system output with manual

inspection of PCB images. The vision system achieved a 92% detection rate across all test PCBs. Missed detections primarily occurred with small components or when copper trace reflections created confusing image features.

False positive rate was approximately 3%, with most false detections occurring at board edges or fiducial marks. Post-processing filters could reduce false positives with minimal impact on true detections. Detection performance improved with consistent lighting and proper PCB positioning. The diffused LED ring light significantly improved detection reliability compared to ambient lighting alone.

5.2.3 Cycle Time

Cycle time measures the total time required to place a component, from image capture to placement completion. Cycle time directly impacts system throughput. Average cycle time was approximately 5 seconds per component, broken down as: image capture (0.5s), image processing (1.0s), coordinate transmission (0.2s), gantry motion (2.5s), and gripper operation (0.8s).

Motion time varied with distance, with shorter moves completing in under 2 seconds and longer moves requiring up to 4 seconds. Optimized motion profiles could reduce these times. Image processing time could be reduced by optimizing the OpenCV pipeline or using a more powerful host computer. Parallel processing of multiple components could further improve throughput.

5.3 TEST RESULTS

Table 5.1: Performance Test Results

Parameter	Target	Achieved	Status
Placement Accuracy	+/-3 mm	+/-2 mm	PASS
Repeatability	>=85%	90%	PASS
Detection Rate	>=85%	92%	PASS
Cycle Time	<=10 sec	5 sec	PASS

Parameter	Target	Achieved	Status
Hardware Cost	<=Rs. 25,000	Rs. 18,000	PASS
PCB Samples Tested	5 boards	5 boards	PASS

5.4 COMPARATIVE ANALYSIS

The proposed system was compared with alternative PCB assembly approaches to evaluate its relative advantages and limitations. The comparison considers cost, accuracy, speed, and suitability for different applications.

Table 5.2 Comparative Analysis

Parameter	Manual Assembly	Industrial SMT	Proposed System
Cost	Low	Very High (Rs.5-20L)	Low (Rs.18K)
Accuracy	Moderate	Very High (+/- 0.05mm)	High (+/- 2mm)
Speed	Slow	Very Fast	Moderate (5s/comp)
Skilled Labor	Highly Required	CAM Engineers	Minimal
Automation Level	None	Fully Automated	Semi-Intelligent
Setup Complexity	Low	Very High	Moderate
Suitable for Prototyping	Yes	No	Yes

Parameter	Manual Assembly	Industrial SMT	Proposed System
Suitable for Mass Production	No	Yes	Limited
Maintenance Cost	Low	High	Low
Infrastructure Required	Minimal	Industrial Setup	Lab-Level Setup

5.5 ERROR ANALYSIS

5.5.1 Sources of Error

Several factors contribute to positioning error in the system. Understanding these error sources helps identify opportunities for improvement. Mechanical backlash in the lead screw nuts creates position uncertainty when reversing direction. Anti-backlash nuts could reduce this error source. Lead screw wobble due to imperfect alignment causes periodic position variation. Improved bearing support and alignment would reduce this effect.

Frame flexure under load causes position-dependent compliance. A more rigid frame design would improve accuracy across the working area. Camera calibration drift occurs due to temperature changes or mechanical disturbance. Regular recalibration maintains coordinate accuracy. Vision processing error from threshold variations or contour detection uncertainty affects coordinate extraction. Improved algorithms could reduce this error.

5.5.2 Error Mitigation

Several strategies can reduce positioning error and improve system accuracy. These improvements can be implemented as system upgrades. Anti-backlash lead screw nuts eliminate position uncertainty from thread clearance. These specialized nuts use spring-loaded split designs to maintain constant thread contact.

Improved frame rigidity through larger extrusion profiles or additional bracing reduces compliance. The trade-off is increased cost and weight. Regular calibration using the checkerboard pattern maintains coordinate accuracy. Automated calibration routines could

simplify this process. Vision algorithm improvements including better threshold selection and contour validation could improve detection accuracy and reduce false positives.

5.6 IMPACT OF PCB ASSEMBLY AUTOMATION

The Vision-Guided Automated PCB Component Mounting System improves PCB assembly by reducing errors, setup time, and manual effort through vision-based automation. It achieves a placement accuracy of ± 2 mm with high repeatability, making it suitable for prototyping and educational applications. With a low cost of Rs. 18,000, the system provides an efficient and affordable alternative to manual assembly while offering a practical platform for learning and small-scale manufacturing.

5.7 DISCUSSION

The test results confirm that the proposed system meets its design objectives by providing automated PCB assembly at low cost. It achieves ± 2 mm placement accuracy, 92% detection rate, and a cycle time of 5 seconds per component, making it suitable for prototyping and educational use. The system effectively bridges the gap between manual assembly and expensive industrial solutions, while offering scope for improvement through mechanical and algorithm enhancements.

5.8 SUMMARY

This chapter has presented the test results and analysis of the Vision-Guided Low-Cost Automated PCB Component Mounting System. Testing in a controlled laboratory environment evaluated placement accuracy, detection rate, and cycle time. The system achieved ± 2 mm placement accuracy with 90% repeatability, 92% component detection rate, and 5-second cycle time per component. All performance targets were met or exceeded. Comparative analysis with manual assembly and industrial SMT systems shows the proposed system fills a gap in affordable automation for prototyping applications. Error analysis identified mechanical backlash, frame flexure, and vision processing as primary error sources, with mitigation strategies proposed. The results demonstrate that low-cost automated PCB assembly is feasible using computer vision and embedded control technologies.

CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

The Vision-Guided Low-Cost Automated PCB Component Mounting System was successfully developed and tested, demonstrating that PCB assembly can be automated at a low cost. By integrating computer vision with embedded control, the system achieves a placement accuracy of ± 2 mm, a detection rate of 92%, and a cycle time of 5 seconds per component. With a total hardware cost of approximately Rs. 18,000, the system provides an efficient and affordable solution for academic, prototyping, and small-scale manufacturing applications, effectively bridging the gap between manual assembly and expensive industrial systems.

6.2 FUTURE SCOPE

The system can be further enhanced by integrating a robotic pick-and-place arm for full automation and supporting SMD components. Implementation of AI-based vision techniques can improve component recognition and eliminate dependency on predefined templates. Additional features such as automatic soldering, conveyor-based PCB handling, and real-time defect detection can enable complete end-to-end automation. Furthermore, improvements in mechanical design, component feeding systems, and Industry 4.0 integration can increase accuracy, efficiency, and scalability of the system.

6.3 SUMMARY

This project has successfully demonstrated the feasibility of low-cost automated PCB assembly using computer vision and embedded control technologies. The Vision-Guided Low-Cost Automated PCB Component Mounting System achieves ± 2 mm placement accuracy with 92% detection rate at a hardware cost under Rs. 20,000. The modular architecture separates vision processing from motion control, enabling independent development and upgrade of each subsystem. The system fills a gap between manual assembly and industrial automation, making automation accessible to academic institutions and small-scale manufacturers. Future enhancements including robotic arm integration, AI-

based classification, and Industry 4.0 connectivity can extend the system's capabilities for broader applications.

REFERENCES

- [1] S. Smith, A. Jones, and R. Patel, "Vision-Guided Robotics in Electronics Assembly," IEEE Transactions on Automation Science and Engineering, vol. 16, no. 3, pp. 1123-1132, 2019.
- [2] R. Kumar and P. Sharma, "Design and Implementation of Low-Cost Pick and Place Machine for PCB Assembly," International Journal of Engineering Research & Technology, vol. 8, no. 3, pp. 245-250, Mar. 2021.
- [3] H. Lee, "Image Processing Framework for PCB Defect Inspection using Edge Detection and Template Matching," International Journal of Electronics Manufacturing, vol. 12, no. 4, pp. 78-86, Nov. 2020.
- [4] Y. Zhang and L. Chen, "Automated Optical Inspection System for PCB Manufacturing," IEEE Transactions on Industrial Electronics, vol. 67, no. 5, pp. 4123-4132, May 2020.
- [5] S. Nakamura and Y. Ito, "Vision-Based Robotic Manipulation for Electronic Assembly," Journal of Smart Manufacturing Systems, vol. 12, no. 2, pp. 89-98, Feb. 2019.
- [6] M. Rodriguez and H. Lee, "Optimization of Surface Mount Technology Assembly Lines," International Journal of Electronics Manufacturing, vol. 15, no. 1, pp. 34-42, Jan. 2022.
- [7] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [8] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.
- [9] Espressif Systems, "ESP8266 Technical Reference Manual," Version 1.7, 2022.
- [10] Arduino, "Arduino UNO Rev3 Technical Specifications," [Online]. Available: <https://www.arduino.cc/en/hardware>.
- [11] MathWorks, "Image Processing Toolbox User Guide," MathWorks, Natick, MA, 2023.
- [12] Hari Sankar S., Jayadev K., Suraj B., and Aparna P., "A Comprehensive Solution to Pick-and-Place Automation and Vision-Based Positioning," in Proc. IEEE Int. Conf. on Embedded Systems, Nov. 2016.
- [13] D. Tran and J. Park, "Vision-Based Coordinate Mapping for Robotic Pick-and-Place Applications," IEEE Access, vol. 10, pp. 33421-33430, 2022.
- [14] P. Singh and V. Sharma, "Design Considerations for Low-Cost Automation Systems in Electronics Manufacturing," in Proc. IEEE Int. Conf. Industrial Engineering and Applications, 2021, pp. 145-150.

- [15] M. A. Khan et al., "Low-Cost Vision-Based Pick and Place Robotic System for Small Scale Industries," in Proc. IEEE Int. Conf. Intelligent Systems and Applications, 2021, pp. 210-215.
- [16] J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Pearson, 2005.
- [17] R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2011.
- [18] K. Fu, R. Gonzalez, and C. Lee, *Robotics: Control, Sensing, Vision, and Intelligence*, McGraw-Hill, 1987.
- [19] B. Siciliano and O. Khatib, *Springer Handbook of Robotics*, Springer, 2016.
- [20] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, Cambridge University Press, 2004.
- [21] D. Forsyth and J. Ponce, *Computer Vision: A Modern Approach*, Pearson, 2012.
- [22] J. Tsai, "A Versatile Camera Calibration Technique for High-Accuracy 3D Machine Vision," IEEE Journal of Robotics and Automation, vol. 3, no. 4, pp. 323-344, 1987.
- [23] Z. Zhang, "A Flexible New Technique for Camera Calibration," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 11, pp. 1330-1334, 2000.
- [24] R. Gonzalez and R. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.
- [25] T. Otsu, "A Threshold Selection Method from Gray-Level Histograms," IEEE Transactions on Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62-66, 1979.
- [26] C. Harris and M. Stephens, "A Combined Corner and Edge Detector," in Proc. Alvey Vision Conf., 1988, pp. 147-151.
- [27] J. Canny, "A Computational Approach to Edge Detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 8, no. 6, pp. 679-698, 1986.
- [28] S. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," International Journal of Computer Vision, vol. 60, no. 2, pp. 91-110, 2004.
- [29] D. Ballard, "Generalizing the Hough Transform to Detect Arbitrary Shapes," Pattern Recognition, vol. 13, no. 2, pp. 111-122, 1981.
- [30] R. Bolles and M. Fischler, "Random Sample Consensus (RANSAC)," Communications of the ACM, vol. 24, no. 6, pp. 381-395, 1981.
- [31] A. Saxena et al., "Robotic Grasping of Novel Objects," International Journal of Robotics Research, vol. 27, no. 2, pp. 157-173, 2008.

- [32] Y. LeCun et al., "Gradient-Based Learning Applied to Document Recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278-2324, 1998.
- [33] K. He et al., "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, 2016, pp. 770-778.
- [34] A. Krizhevsky et al., "ImageNet Classification with Deep Convolutional Neural Networks," in Proc. NIPS, 2012.
- [35] S. Ren et al., "Faster R-CNN: Towards Real-Time Object Detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017.

APPENDIX

MATHEMATICAL ANALYSIS

A.1 Linear Step Resolution

The theoretical linear resolution of the positioning system is calculated as follows:

$$R = P / (N \times M)$$

Where:

R = Linear resolution (mm/step)

P = Lead screw pitch = 2 mm

N = Motor steps per revolution = 200

M = Microstepping factor = 16

$$R = 2 / (200 \times 16) = 0.000625 \text{ mm/step}$$

A.2 Total Positioning Error

The total positioning error is estimated using root-sum-square combination of error sources:

$$E_{\text{total}} = \sqrt{E_{\text{motor}}^2 + E_{\text{mechanical}}^2 + E_{\text{vision}}^2}$$

Where:

E_motor = Step quantization error = 0.000625 mm

E_mechanical = Backlash and flexure = 1.5 mm

E_vision = Centroid detection error = 0.5 mm

$$E_{\text{total}} = \sqrt{0.000625^2 + 1.5^2 + 0.5^2} = 1.58 \text{ mm}$$

A.3 Steps Required for Displacement

The number of steps required to move a distance D is calculated as:

$$\text{Steps} = D \times N \times M / P = D \times 200 \times 16 / 2 = D \times 1600$$

Example: To move 10 mm, Steps = 10 x 1600 = 16,000 steps

SYSTEM SPECIFICATIONS

Table 6.1: System Specification

Parameter	Specification
Working Area	200 x 200 mm
Positioning Accuracy	+/- 2 mm
Repeatability	90%
Detection Rate	92%
Cycle Time	5 seconds per component
Maximum Speed	40 mm/s
Power Consumption	30 W
Input Voltage	12V DC
Communication	UART 115200 bps
Camera Resolution	8 MP (3264 x 2448)
Motor Type	NEMA17 Stepper
Driver Type	A4988 Microstepping
Microstepping	1/16 step
Lead Screw Pitch	2 mm
Total Hardware Cost	Rs. 18,000

COMPONENT DATASHEETS

Datasheets for major components are available from manufacturers:

ESP8266 NodeMCU: https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf

A4988 Stepper Driver:
https://www.pololu.com/file/0J450/a4988_DMOS_microstepping_driver_with_translator.pdf

NEMA17 Stepper Motor: <https://www.omc-stepperonline.com/download/17HS19-2004S1.pdf>

LM2596 Buck Converter: <https://www.ti.com/lit/ds/symmlink/lm2596.pdf>